LINUX INTERVIEW PREPARATION GUIDE

LAB WORKBOOK

LABS

Linux Lab Exercises

Hands-on labs for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience) · Ubuntu, RHEL and AWS

IN THIS WORKBOOK

00 Lab Environment Setup
01 Linux Fundamentals
02 Users, Groups and Permissions
03 Packages and Services
04 Storage and Filesystems
05 Networking and SSH
06 Bash Scripting and Automation
07 Log Analysis and Monitoring
08 Performance Troubleshooting
09 Docker and Kubernetes
10 AWS EC2 Administration
11 Web Application Deployment
12 CI/CD Release and Rollback
13 Capstone Project
14 Final Assessment
15 Takeaways and Next Steps

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

WORKBOOK OBJECTIVE

This workbook turns Chapters 1–18 into hands-on practice. You will build, break and fix a real Linux server: users, services, storage, networking, scripts, logs, performance, containers, AWS EC2, web deployments and a CI/CD release with rollback. The goal is simple: do the work without a tutorial, and explain every step in an interview.

0 Lab Environment Setup

How to use this workbook

Every lab follows the same loop. Do not copy and paste blindly. Type the commands, read the output and ask yourself "why did that happen?"

1. **Read** the objective and the scenario.
2. **Do** the tasks yourself.
3. **Verify** your results against the expected output.
4. **Break and fix** things with the troubleshooting challenges.
5. **Clean up** what the lab created.
6. **Explain** what you did, as if you were answering an interview question.

Each lab starts with an "at a glance" table. It shows the level, the chapters the lab revises, the best environment and the expected time. The orange number on each heading is the lab number.

PRODUCTION WARNING

Run these labs only on a test VM or in a test AWS account. Several labs format disks, edit `/etc/fstab`, change firewall and SSH settings, stop services and generate CPU and memory load. Never practise on production servers, shared servers or real data.

DEVOPS NOTE

Take a VM snapshot before Labs 2, 4, 5 and 8. On AWS, create an EBS snapshot or an AMI. If you break something, you can revert in seconds instead of rebuilding. Real teams test risky changes the same way.

Choose your environment

Environment	Good for	Limitations
Ubuntu Server VM	All labs except Lab 10 (main path)	Needs a hypervisor (VirtualBox, VMware, Hyper-V, UTM, Multipass)
Rocky Linux / AlmaLinux VM	RHEL-family practice: `dnf`, firewalld, SELinux	Some extra tools need the EPEL repository
AWS EC2 (Ubuntu or Amazon Linux 2023)	Lab 10, and cloud versions of every other lab	Costs money; terminate resources when you finish
WSL (Ubuntu)	Labs 1, 2, 6 and 7 basics	No real disks, boot process or firewall; systemd may need enabling
Docker container	Quick command practice	No systemd, real networking or storage; changes disappear

Recommended VM specification

- Ubuntu Server 24.04 LTS or a newer LTS release (the main path in this workbook).
- Optional second VM: Rocky Linux or AlmaLinux 9/10, or Amazon Linux 2023, to practise RHEL-family differences.
- 2 vCPU, 4 GB RAM, 25 GB disk, internet access, and SSH access from your laptop.

> **NOTE**
> Commands are written for Ubuntu first. Where Ubuntu and RHEL-family systems differ, a side-by-side table or a second code block shows both. In this workbook, **RHEL-family** means RHEL, Rocky Linux and AlmaLinux. Amazon Linux 2023 differences are called out by name.

Install the lab toolkit

Ubuntu / Debian

```
sudo apt update
sudo apt install -y \
    curl wget git vim nano tree jq unzip lsof ncdu \
    dnsutils net-tools sysstat tcpdump traceroute netcat-openbsd \
    apache2-utils openssh-server psmisc acl
```

RHEL / Rocky / AlmaLinux / Amazon Linux 2023

```
# Rocky / AlmaLinux only: enable EPEL for extra tools such as ncdu
sudo dnf install -y epel-release

sudo dnf install -y \
    wget git vim nano tree jq unzip lsof \
    bind-utils net-tools sysstat tcpdump traceroute nmap-ncat \
    httpd-tools openssh-server psmisc acl
```

Nginx is not in this list on purpose. You install it yourself in Lab 3. `curl` is already installed on RHEL-family systems; on Amazon Linux 2023 do not replace `curl-minimal`. If a package is not found on Amazon Linux 2023 (it has no EPEL), skip it. Every lab gives an alternative.

Tool or service	Ubuntu	RHEL-family
`dig`, **`nslookup`**	`dnsutils`	`bind-utils`
`nc`	`netcat-openbsd`	`nmap-ncat`
`htpasswd`, **`ab`**	`apache2-utils`	`httpd-tools`
SSH service name	`ssh`	`sshd`
Cron service name	`cron`	`crond` (package `cronie`)

Check your environment

```
hostname
cat /etc/os-release
uname -a
nproc
free -h
df -h
```

Create the practice directory

```
mkdir -p ~/linux-labs/{scripts,logs,backups,projects}
cd ~/linux-labs
```

Use this directory for every exercise unless a lab says otherwise.

Placeholders used in this workbook

Placeholder	Replace with
`SERVER_IP`	The IP address of your lab server (`hostname -I`)
`USERNAME`	Your Linux username (`whoami`)
`/dev/loopX`	The loop device returned by `losetup` in Lab 4
`PID`	A real process ID (`pgrep` or `systemctl show -p MainPID`)
`BUCKET` , `vol-…` , `i-…`	Your own AWS bucket name, volume ID and instance ID

1 Linux Fundamentals and the Command Line

Lab	Level	Revises	Best environment	Time
Lab 1	Beginner	Chapters 1–2	Any Linux, including WSL	45 min

> **LAB OBJECTIVE**
>
> Get comfortable on the command line. Identify the system, explore the filesystem hierarchy, manage files and understand links and inodes.
>
> **Scenario:** You have joined a company as a Junior Linux Administrator. Your first task is to explore a server and organise the files of a new application.

Task 1: Identify the system

```
pwd
whoami
id
hostname
hostnamectl          # hostname, OS, kernel and virtualization in one view
date
uptime
uname -r             # kernel version only
cat /etc/os-release
```

Write down the username, hostname, distribution and version, kernel version and uptime. Check the `Virtualization` line in `hostnamectl` to see whether you are on a VM.

Task 2: Explore the filesystem hierarchy

```
ls /
ls -ld /etc /var /home /tmp /usr /opt /root
ls /root             # Permission denied for a normal user
sudo ls /root
```

Directory	What it holds	Where you meet it in DevOps
`/etc`	System and service configuration	`/etc/nginx/nginx.conf`, `/etc/ssh/sshd_config`, `/etc/fstab`
`/var`	Variable data: logs, caches, spools, web roots	`/var/log`, `/var/lib/docker`, `/var/www`
`/home`	Users' home directories	A deploy user's SSH keys in `/home/deploy/.ssh`
`/tmp`	Temporary files; world-writable with the sticky bit	Build scratch space; may be cleaned at reboot
`/usr`	Installed programs, libraries and shared data	`/usr/bin`, `/usr/sbin`, `/usr/share/nginx` (RHEL)
`/opt`	Optional and third-party software	Vendor agents and self-contained applications
`/root`	The root user's home directory	Not the same as `/`; not readable by normal users

Task 3: Create an application directory structure

```
mkdir -p ~/linux-labs/projects/myapp/{config,logs,backup,scripts}
tree ~/linux-labs/projects/myapp
```

Expected:

```
myapp/
├── backup/
├── config/
├── logs/
└── scripts/

4 directories, 0 files
```

Task 4: Practise file operations

```
cd ~/linux-labs/projects/myapp

touch config/app.conf logs/app.log backup/backup.txt
cp config/app.conf config/app.conf.bak
mv backup/backup.txt backup/old-backup.txt
ls -lahR
```

Task 5: Search for files

```
find ~/linux-labs -name "*.log"
find ~/linux-labs -type d
find ~/linux-labs -type f
find ~/linux-labs -type f -mmin -30      # changed in the last 30 minutes
find ~/linux-labs -type f -empty         # empty files
```

Task 6: File content and redirection

```
echo "Application started" > logs/app.log     # >  creates or OVERWRITES
echo "Database connected" >> logs/app.log     # >> APPENDS
cat logs/app.log
tail -n 1 logs/app.log

ls /does-not-exist > logs/out.txt 2>&1     # send errors to the same file
cat logs/out.txt
```

WARNING

A single `>` typed instead of `>>` silently empties a file. In production, this can wipe a config file or a log. Bash's `set -o noclobber` makes `>` refuse to overwrite an existing file.

Task 7: Hard links, symbolic links and inodes

```
cd ~/linux-labs/projects/myapp
echo "v1" > config/app.conf
ln config/app.conf config/app-hard.conf        # hard link: same inode
ln -s app.conf config/app-soft.conf            # symlink: stores a path
ls -li config/
```

Expected (inode numbers will differ):

```
1310724 -rw-r--r-- 2 ubuntu ubuntu 3 Nov  5 10:20 app.conf
1310724 -rw-r--r-- 2 ubuntu ubuntu 3 Nov  5 10:20 app-hard.conf
1310731 lrwxrwxrwx 1 ubuntu ubuntu 8 Nov  5 10:20 app-soft.conf → app.conf
```

The two files share one inode, and the link count is `2`. Now delete the original file:

```
rm config/app.conf
cat config/app-hard.conf     # still prints v1: the data has another name
cat config/app-soft.conf     # No such file or directory: dangling symlink
echo "v1" > config/app.conf  # the symlink works again...
ls -li config/               # ...but app.conf is a NEW inode, separate from app-hard.con
```

Troubleshooting challenges

Challenge 1 The misplaced file

Setup: `touch ~/linux-labs/logs/app.conf.backup`. Now pretend you do not know where it is.

1. Find it with `find` using a name pattern.
2. Rename it to `old-app.conf`.
3. Move it to `myapp/backup/` and keep a copy in `myapp/config/`.

Challenge 2 Where did the error go?

Run both commands, then explain why only one of them captures the error in the file:

```
ls /nope > out1.txt 2>&1
ls /nope 2>&1 > out2.txt
```

Hint: redirections are processed left to right. In the second command, stderr is pointed at the terminal before stdout moves to the file.

INTERVIEW TRAP

"Can you hard-link a directory, or hard-link across filesystems?" No. A hard link points to an inode, and inode numbers are only unique within one filesystem. Use a symbolic link for directories and for links across filesystems.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Absolute vs relative paths, and the purpose of `/etc`, `/var`, `/opt` and `/tmp`.
- Hard link vs symbolic link, and what an inode stores.
- `cp` vs `mv`. Within one filesystem, `mv` is only a rename, so the inode does not change.
- `>` vs `>>` vs `2>&1`.

You are done when

- ✓ You can describe the server (OS, kernel, uptime) in under a minute.
- ✓ You can predict what happens to hard links and symlinks when the original file is deleted.
- ✓ You can find any file by name, type or modification time.

2 Users, Groups and Permissions

Lab	Level	Revises	Best environment	Time
Lab 2	Beginner–Intermediate	Chapter 3 (sudo hardening: Chapter 9)	VM or EC2	60 min

LAB OBJECTIVE

Manage users, groups, ownership, permission bits, shared team directories and restricted sudo access.

Scenario: Your company has three teams: Developers, DevOps and QA. Each team needs its own shared directory. Other teams must not be able to access it.

Task 1: Create users

```
sudo useradd -m -s /bin/bash developer1
sudo useradd -m -s /bin/bash devops1
sudo useradd -m -s /bin/bash qa1

sudo passwd developer1      # repeat for devops1 and qa1
id developer1
```

On Ubuntu, `useradd` without `-s` gives the user `/bin/sh`. RHEL-family systems default to `/bin/bash`. Ubuntu also has the interactive `adduser` wrapper.

Task 2: Create groups and add members

```
sudo groupadd developers
sudo groupadd devops
sudo groupadd qa

sudo usermod -aG developers developer1
sudo usermod -aG devops devops1
sudo usermod -aG qa qa1

id developer1
```

Expected (IDs will differ):

```
uid=1001(developer1) gid=1001(developer1) groups=1001(developer1),1004(developers)
```

> **WARNING**
>
> `usermod -G developers user` without `-a` **replaces** all of the user's supplementary groups. Admins have locked themselves out of `sudo` or `wheel` this way. Always use `-aG`.

Task 3: Create team directories with setgid

```
sudo mkdir -p /company/{developers,devops,qa}

sudo chown root:developers /company/developers
sudo chown root:devops     /company/devops
sudo chown root:qa         /company/qa

sudo chmod 2770 /company/developers /company/devops /company/qa
ls -ld /company/*
```

Expected:

```
drwxrws--- 2 root developers 4096 Nov  5 10:30 /company/developers
drwxrws--- 2 root devops     4096 Nov  5 10:30 /company/devops
drwxrws--- 2 root qa         4096 Nov  5 10:30 /company/qa
```

The `s` in the group position is the **setgid** bit (the leading `2` in `2770`). New files created inside inherit the directory's group, not the creator's primary group.

Task 4: Test access

```
sudo -u developer1 touch /company/developers/dev-file.txt
ls -l /company/developers/        # group is "developers", thanks to setgid
sudo -u qa1 ls /company/developers
```

Expected:

```
ls: cannot open directory '/company/developers': Permission denied
```

Task 5: The umask trap in shared directories

```
sudo useradd -m -s /bin/bash -G developers developer2
sudo -u developer2 sh -c 'echo "edit" >> /company/developers/dev-file.txt'   # Permission
ls -l /company/developers/dev-file.txt                                       # -rw-r--r--:
```

Setgid fixes the group, but the default umask (`022`) still creates files without group write. Fix it for new files with a default ACL, and fix the existing file once:

```
sudo setfacl -d -m g::rwx /company/developers     # default ACL for new files
sudo chmod g+w /company/developers/dev-file.txt   # fix the existing file
getfacl /company/developers

sudo -u developer1 touch /company/developers/new-file.txt
ls -l /company/developers/new-file.txt            # now -rw-rw----
```

Task 6: Permission values and special bits

```
cd ~/linux-labs
touch test.sh config.txt secret.txt
chmod 755 test.sh
chmod 644 config.txt
chmod 600 secret.txt
stat -c '%a %A %n' test.sh config.txt secret.txt

ls -l /usr/bin/passwd    # SUID: -rwsr-xr-x
ls -ld /tmp              # sticky bit: drwxrwxrwt
```

Special bit	Octal	On a file	On a directory
SUID	4000	Runs as the file's owner (`passwd`)	No common effect on Linux
SGID	2000	Runs with the file's group	New files inherit the directory's group
Sticky	1000	No common effect on Linux	Only a file's owner can delete it (`/tmp`)

Task 7: Restricted sudo with a drop-in file

```
sudo visudo -f /etc/sudoers.d/devops1
```

Add one line, then save:

```
devops1 ALL=(root) /usr/bin/systemctl status nginx, /usr/bin/systemctl restart nginx
```

```
sudo chmod 0440 /etc/sudoers.d/devops1
sudo -l -U devops1
```

Item	Ubuntu	RHEL-family
Full admin group	`sudo`	`wheel`
Drop-in directory	`/etc/sudoers.d/`	`/etc/sudoers.d/`
Auth log	`/var/log/auth.log`	`/var/log/secure`

INTERVIEW TRAP

"Is `devops1 ALL=(root) /usr/bin/vim` a safe restricted rule?" No. Inside vim, `:!bash` opens a root shell. Pagers (`less`), editors and many interpreters allow shell escapes. Grant exact commands, and use `sudoedit` for file editing.

Troubleshooting challenges

Challenge 1 Deployment user cannot write

Setup: `sudo mkdir -p /var/www/lab2-app && sudo chmod 700 /var/www/lab2-app`. Then `sudo -u devops1 touch /var/www/lab2-app/x` fails. Investigate:

```
id devops1
ls -ld /var/www/lab2-app
namei -l /var/www/lab2-app
```

Fix it with group ownership: `sudo chgrp devops /var/www/lab2-app && sudo chmod 2775 /var/www/lab2-app`. Never use `chmod 777`.

Challenge 2 "I added myself to the group but it still says denied"

Add your own user to `devops` with `usermod -aG`, then run `id` in your current SSH session. The new group is missing. Group membership is read at login. Log out and back in (or run `newgrp devops`) and check again.

WARNING

Cleanup deletes users, their home directories and directories. Read every path before you press Enter.

```
sudo rm -f /etc/sudoers.d/devops1
for u in developer1 developer2 devops1 qa1; do sudo userdel -r "$u"; done
sudo groupdel developers; sudo groupdel devops; sudo groupdel qa
sudo rm -rf /company /var/www/lab2-app
```

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- UID, GID, primary vs supplementary groups.
- Permission bits, umask, and why `chmod 777` is dangerous.
- SUID, SGID and the sticky bit, with one real example each.
- How you would give a team a shared directory and a deploy user limited sudo.

You are done when

- ✓ Each team can write only in its own directory, and new files get the right group and mode.
- ✓ You can read `namei -l` output to find which directory blocks access.

✓ You can write and validate a restricted sudoers drop-in.

3 Package Management and Service Administration

Lab	Level	Revises	Best environment	Time
Lab 3	Beginner–Intermediate	Chapters 4–5	VM or EC2 (not WSL without systemd)	60 min

LAB OBJECTIVE

Install software with the package manager, manage a systemd service, read its logs and safely recover from a broken configuration.

Scenario: You need to prepare a Linux server to host a website with Nginx.

Task 1: Refresh repositories and review updates

Action	Ubuntu	RHEL-family
Refresh metadata	`sudo apt update`	`sudo dnf makecache`
List available updates	`apt list --upgradable`	`dnf check-update`
Security updates only	`unattended-upgrades` / `apt` pinning	`dnf updateinfo list --security`
Apply updates	`sudo apt upgrade`	`sudo dnf upgrade`

INTERVIEW TRAP

`dnf check-update` exits with code **100** when updates are available. That is not an error. Scripts that treat any non-zero exit code as a failure will break on it.

Task 2: Install Nginx and inspect the package

Action	Ubuntu	RHEL-family
Install	`sudo apt install -y nginx`	`sudo dnf install -y nginx`
Package details	`apt show nginx`	`dnf info nginx`
Files in the package	`dpkg -L nginx-core`	`rpm -ql nginx`
Which package owns a file	`dpkg -S /usr/sbin/nginx`	`rpm -qf /usr/sbin/nginx`
After install	Started and enabled automatically	**Not** started, **not** enabled

On newer Ubuntu releases the binary may come from the `nginx` package itself rather than `nginx-core`. Use `dpkg -S /usr/sbin/nginx` to find the right package name. Check the version with `nginx -v`.

Task 3: Manage the service

```
sudo systemctl enable --now nginx    # enable at boot AND start now
systemctl is-active nginx            # active
systemctl is-enabled nginx           # enabled
systemctl status nginx --no-pager
```

On RHEL-family systems with firewalld running, also open HTTP:

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

Amazon Linux 2023 has no host firewall by default. Access is controlled by the EC2 security group (Lab 10).

Task 4: Test the website

```
curl -I http://localhost
```

Expected (version and OS label will differ):

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Content-Type: text/html
Connection: keep-alive
```

Task 5: Serve a custom page

Item	Ubuntu	RHEL-family
Default web root	`/var/www/html`	`/usr/share/nginx/html`
Site config	`/etc/nginx/sites-available/` + symlink in `sites-enabled/`	`/etc/nginx/conf.d/*.conf`
Worker user	`www-data`	`nginx`

```
echo "Welcome to my Linux Lab" | sudo tee /var/www/html/index.html
# RHEL: sudo tee /usr/share/nginx/html/index.html
curl http://localhost
```

Task 6: Read the service logs

```
sudo journalctl -u nginx -n 50 --no-pager
sudo tail -n 50 /var/log/nginx/error.log
systemctl list-units --type=service --state=failed
```

Task 7: Stop, observe, start

```
sudo systemctl stop nginx
systemctl status nginx --no-pager    # inactive (dead)
curl -I http://localhost             # Failed to connect ... Connection refused
sudo systemctl start nginx
```

Task 8: Break the configuration safely

Use a separate test file in `conf.d`. Never edit the main `nginx.conf` for this exercise.

```
echo "this_is_not_valid;" | sudo tee /etc/nginx/conf.d/broken-lab.conf
sudo nginx -t
```

Expected:

```
nginx: [emerg] unknown directive "this_is_not_valid" in /etc/nginx/conf.d/broken-lab.conf
nginx: configuration file /etc/nginx/nginx.conf test failed
```

Now compare reload and restart with the broken file still in place:

```
sudo systemctl reload nginx     # fails or logs an error, but the OLD config keeps servin
curl -I http://localhost        # still 200 OK

sudo systemctl restart nginx    # fails: Nginx is now DOWN
curl -I http://localhost        # Connection refused

sudo rm /etc/nginx/conf.d/broken-lab.conf
sudo nginx -t && sudo systemctl start nginx
```

INTERVIEW INSIGHT

Depending on the unit file, `reload` either fails visibly or "succeeds" while Nginx logs the error and keeps the old configuration. Either way, the site stays up. A `restart` with a broken configuration takes the site down. In production, always run `nginx -t && systemctl reload nginx`.

Troubleshooting challenge

Challenge 1 Nginx will not start: port already in use

Setup: `sudo systemctl stop nginx`, then in a second terminal run `sudo python3 -m http.server 80`. Now try `sudo systemctl start nginx`.

```
systemctl status nginx --no-pager
sudo journalctl -u nginx -n 20 --no-pager   # bind() to 0.0.0.0:80 failed (98: Addres
sudo ss -tlnp 'sport = :80'                 # shows python3 owning the port
```

Stop the Python server with Ctrl+C, then start Nginx again.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- What a package manager does: dependencies, repositories and signatures.
- `enable` vs `start` (and `enable --now`), and `restart` vs `reload`.
- How you investigate a service that fails to start: `status`, `journalctl -u`, config test, port check.
- Why you test configuration before reloading in production.

You are done when

- ✓ Nginx serves your custom page and starts at boot.
- ✓ You can show, with commands, why `reload` is safer than `restart` after a bad config change.

✓ You can find which process owns a port.

4 Storage, Partitions and Filesystem Operations

Lab	Level	Revises	Best environment	Time
Lab 4	Intermediate	Chapter 6	VM or EC2 (not WSL or containers)	90 min

LAB OBJECTIVE

Work with block devices, filesystems, mounts and `/etc/fstab`. Reproduce the classic storage incidents safely: disk full, inode exhaustion, deleted-but-open files and "device is busy".

Scenario: A new application needs extra storage for uploads and backups. Before you touch a real disk, you practise on a safe loop device.

Task 1: Inspect current storage

```
lsblk -f          # devices, filesystems, UUIDs, mount points
df -hT            # usage with filesystem type
df -i             # inode usage
findmnt           # mount tree
sudo fdisk -l
```

Identify the root filesystem and its type, the free space, and every mounted device. Ubuntu usually uses **ext4** for root; RHEL-family systems use **XFS**.

Task 2: Create a safe test disk

```
dd if=/dev/zero of=~/linux-labs/disk1.img bs=1M count=512 status=progress
LOOP=$(sudo losetup --find --show ~/linux-labs/disk1.img)
echo "$LOOP"      # e.g. /dev/loop0 - write it down
losetup -l
```

Keep this terminal open. If you open a new one, set the variable again, for example `LOOP=/dev/loop0`.

Task 3: Create a filesystem

> **WARNING**
>
> `mkfs` destroys all data on the target device. First prove that the device is your empty loop device: `lsblk -f "$LOOP"` must show no `FSTYPE`, and `sudo blkid "$LOOP"` must print nothing.

```
sudo mkfs.ext4 -L LABDATA "$LOOP"
# XFS alternative (RHEL default): sudo mkfs.xfs -L LABDATA "$LOOP"
```

Use ext4 for this lab. The inode exercise in Task 7 needs it, because XFS allocates inodes dynamically.

Task 4: Mount and use it

```
sudo mkdir -p /mnt/linux-lab
sudo mount "$LOOP" /mnt/linux-lab
sudo chown "$USER": /mnt/linux-lab
findmnt /mnt/linux-lab
touch /mnt/linux-lab/test.txt
mkdir /mnt/linux-lab/backups
du -sh /mnt/linux-lab
```

Expected:

```
TARGET         SOURCE     FSTYPE OPTIONS
/mnt/linux-lab /dev/loop0 ext4   rw,relatime
```

Task 5: Make the mount persistent with fstab

```
sudo cp /etc/fstab /etc/fstab.bak-lab4
UUID=$(sudo blkid -s UUID -o value "$LOOP")
echo "UUID=$UUID /mnt/linux-lab ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab

sudo umount /mnt/linux-lab
sudo mount -a                # mounts everything in fstab; errors show here, not at boot
sudo findmnt --verify        # checks fstab syntax and targets
findmnt /mnt/linux-lab
```

PRODUCTION WARNING

A bad `/etc/fstab` line can drop the server into emergency mode at the next boot. On a cloud VM, you then need the serial console to recover. Always back up fstab, use UUIDs, add `nofail` for non-root data disks, and run `mount -a` and `findmnt --verify` before any reboot. Here, the loop device will not exist after a reboot; `nofail` is what lets the boot continue.

Task 6: Fill the disk

```
dd if=/dev/zero of=/mnt/linux-lab/bigfile bs=1M count=600 status=progress
# dd: error writing ... No space left on device
df -h /mnt/linux-lab
sudo tune2fs -l "$LOOP" | grep -i "reserved block count"
rm /mnt/linux-lab/bigfile
```

ext4 reserves about 5% of blocks for root. A normal user sees "disk full" while root can still write. That reserve is what keeps root able to log in and clean up.

Task 7: Exhaust the inodes

```
df -i /mnt/linux-lab                      # roughly 32768 inodes on 512 MB
mkdir /mnt/linux-lab/many
for i in $(seq 1 33000); do : > /mnt/linux-lab/many/f$i; done 2>/dev/null
df -h /mnt/linux-lab                      # plenty of space left...
df -i /mnt/linux-lab                      # ...but IUse% is 100%
touch /mnt/linux-lab/one-more             # No space left on device
rm -rf /mnt/linux-lab/many
```

Expected (numbers will differ slightly):

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0      488M  2.1M  450M   1% /mnt/linux-lab

Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0      32768 32768     0  100% /mnt/linux-lab
```

Task 8: Deleted file still using space

```
dd if=/dev/zero of=/mnt/linux-lab/held.log bs=1M count=200
sleep 3600 < /mnt/linux-lab/held.log &    # a process holds the file open
rm /mnt/linux-lab/held.log
df -h /mnt/linux-lab                       # 200 MB still used
sudo lsof +L1 | grep linux-lab             # deleted file, link count 0
kill %1
df -h /mnt/linux-lab                       # space is back
```

In production, restart the process that holds the file. If you cannot restart it, truncate the file through the process's file descriptor: `: > /proc/PID/fd/N`.

Task 9: "Target is busy"

```
cd /mnt/linux-lab
sudo umount /mnt/linux-lab        # umount: /mnt/linux-lab: target is busy.
sudo fuser -vm /mnt/linux-lab     # shows your own shell
cd ~
sudo umount /mnt/linux-lab        # works
sudo mount /mnt/linux-lab         # remount from fstab for the next task
```

Avoid `umount -l` (lazy unmount) as a habit. It hides the problem: processes keep writing to a filesystem that is no longer visible.

Optional extension: LVM on loop devices

```
truncate -s 512M ~/linux-labs/pv1.img ~/linux-labs/pv2.img
L1=$(sudo losetup -f --show ~/linux-labs/pv1.img)
L2=$(sudo losetup -f --show ~/linux-labs/pv2.img)

sudo pvcreate "$L1" "$L2"
sudo vgcreate labvg "$L1"
sudo lvcreate -n lablv -L 300M labvg
sudo mkfs.ext4 /dev/labvg/lablv
sudo mkdir -p /mnt/lvm-lab && sudo mount /dev/labvg/lablv /mnt/lvm-lab

sudo vgextend labvg "$L2"                      # add the second disk
sudo lvextend -r -L +400M /dev/labvg/lablv     # -r grows the filesystem too
df -h /mnt/lvm-lab
```

Troubleshooting challenges

Symptom	Investigate with	Typical cause
Device not visible	`lsblk`, `dmesg | tail`	Not attached, wrong VM or AZ, needs a rescan
"mount point does not exist"	`ls -ld /path`	Directory was never created
"wrong fs type, bad superblock"	`blkid`, `lsblk -f`	No filesystem yet, or wrong `-t` type
Target is busy	`fuser -vm`, `lsof +D`	A shell or process is using the mount
Disk 100% but `du` shows less	`lsof +L1`	Deleted files held open, or files hidden under a mount point
"No space" with free space shown	`df -i`	Inode exhaustion (many small files)

Challenge 1 **Files hidden under a mount point**

Unmount `/mnt/linux-lab`. Create a 50 MB file in the now-empty `/mnt/linux-lab` directory, which lives on the root filesystem. Mount again. The file "disappears", but root-filesystem usage stays higher. Explain why `du` cannot see it, and how you would find it (unmount, or bind-mount `/` elsewhere and inspect).

WARNING

Clean up in this order: remove the fstab line first, then unmount, then detach. Confirm the line you remove is the lab line.

```
sudo sed -i '\|/mnt/linux-lab|d' /etc/fstab
diff /etc/fstab.bak-lab4 /etc/fstab && echo "fstab restored"
sudo umount /mnt/linux-lab
sudo losetup -d "$LOOP"
# LVM extension only:
sudo umount /mnt/lvm-lab; sudo lvremove -y labvg/lablv; sudo vgremove labvg
sudo pvremove "$L1" "$L2"; sudo losetup -d "$L1" "$L2"
rm ~/linux-labs/disk1.img ~/linux-labs/pv1.img ~/linux-labs/pv2.img
```

INTERVIEW TRAP

"`df` says 100% but `du` only finds 40%. Why?" Usually a large log was deleted while a process still has it open (`lsof +L1`). Less often, files are hidden under a mount point. The fix is to restart or signal the process, not to delete more files.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Block device vs partition vs filesystem vs mount point.
- Every field of an `/etc/fstab` line, and why to use UUIDs and `nofail`.
- Disk space exhaustion vs inode exhaustion, and how to find each.
- How to grow a filesystem: ext4 with `resize2fs`, XFS with `xfs_growfs` (XFS cannot shrink).
- EBS vs EFS, with the AWS details in Lab 10.

5 Networking and SSH Troubleshooting

Lab	Level	Revises	Best environment	Time
Lab 5	Intermediate	Chapter 7 (SSH hardening: Chapter 9)	VM plus your laptop, or EC2	75 min

LAB OBJECTIVE

Read interface and routing information, debug DNS, test ports, tell "refused" from "timed out", set up key-based SSH and manage the host firewall without locking yourself out.

Scenario: A remote application server is not reachable. You must find out which layer is failing.

Task 1: Interfaces and routes

```
ip -br addr               # brief view: interface, state, IPs
ip route                  # look for the "default via" line
ip route get 8.8.8.8      # which interface and gateway this traffic would use
hostname -I
```

Task 2: Connectivity and DNS

```
ping -c 4 8.8.8.8
dig google.com +short
getent hosts google.com
cat /etc/resolv.conf
resolvectl status          # Ubuntu (systemd-resolved)
```

On Ubuntu, `/etc/resolv.conf` usually shows `127.0.0.53`. That is the local systemd-resolved stub, not the real upstream server; `resolvectl status` shows the real one. A failed ping does not prove a host is down. Many networks, and many AWS security groups, block ICMP.

Task 3: /etc/hosts vs DNS

```
echo "10.10.10.10 lab-test.example.com" | sudo tee -a /etc/hosts
getent hosts lab-test.example.com     # 10.10.10.10 - follows /etc/nsswitch.conf (files,
dig +short lab-test.example.com       # nothing - dig asks DNS directly and ignores /etc/
grep ^hosts /etc/nsswitch.conf
sudo sed -i '/lab-test.example.com/d' /etc/hosts
```

> **INTERVIEW TRAP**
>
> **"`dig` returns the right IP but the application connects to the wrong one. Why?"**
> Applications resolve names through the system resolver (`nsswitch.conf`), which checks `/etc/hosts` first. `dig` bypasses it. Always test with `getent hosts` too.

Task 4: Listening ports

```
sudo ss -tulnp
sudo ss -tlnp 'sport = :80'
```

Expected (simplified):

```
Netid State  Local Address:Port  Process
tcp   LISTEN 0.0.0.0:80          users:(("nginx",pid=1234,fd=6))
tcp   LISTEN 0.0.0.0:22          users:(("sshd",pid=812,fd=3))
tcp   LISTEN 127.0.0.1:3000      users:(("node",pid=2210,fd=18))
```

`0.0.0.0` means "all IPv4 interfaces". `127.0.0.1` means "this machine only", so remote clients can never reach that port.

Task 5: Refused vs timed out

```
nc -zv localhost 80                # succeeded
nc -zv localhost 81                # refused: host answered, nothing listening
nc -zv -w 3 10.255.255.1 22        # timed out: no answer at all
curl -v http://localhost
timeout 3 bash -c '</dev/tcp/127.0.0.1/80' && echo open   # works without nc
```

Expected:

```
Connection to localhost (127.0.0.1) 80 port [tcp/http] succeeded!
nc: connect to localhost (127.0.0.1) port 81 (tcp) failed: Connection refused
nc: connect to 10.255.255.1 port 22 (tcp) timed out: Operation now in progress
```

Result	What it means	Look at
Connection refused	Packet reached the host; nothing listening (or a REJECT rule)	Service status, port, bind address
Timed out	Packet silently dropped or lost on the way	Security group, NACL, host firewall, routes
No route to host	Routing or ARP failure, or an ICMP reject	Routes, subnet, gateway

Task 6: Key-based SSH

Item	Ubuntu	RHEL-family
Service	`ssh` (24.04+: socket-activated via `ssh.socket`)	`sshd`
Status	`systemctl status ssh`	`systemctl status sshd`
Auth log	`/var/log/auth.log`	`/var/log/secure`

On your **laptop** (the client):

```
ssh-keygen -t ed25519 -C "linux-lab"
ssh-copy-id -i ~/.ssh/id_ed25519.pub USERNAME@SERVER_IP
ssh USERNAME@SERVER_IP
ssh -v USERNAME@SERVER_IP      # verbose: shows which key is offered and why it fails
```

On the server, check permissions. With `StrictModes` on, sshd ignores keys if these are too open:

```
ls -ld ~/.ssh                  # drwx------ (700)
ls -l ~/.ssh/authorized_keys   # -rw------- (600)
```

Task 7: Disable password login safely

> **WARNING**
>
> Keep your current SSH session open. Test the change from a **second** terminal before you close the first. If key login does not work, you can still revert from the open session.

```
sudo tee /etc/ssh/sshd_config.d/10-lab-hardening.conf >/dev/null <<'EOF'
PasswordAuthentication no
PermitRootLogin no
EOF
sudo sshd -t                                  # validate: no output means OK
sudo sshd -T | grep -Ei '^(passwordauthentication|permitrootlogin)'
sudo systemctl reload ssh                     # RHEL: sudo systemctl reload sshd
```

> **INTERVIEW TRAP**
>
> For most sshd options, **the first value read wins**. Drop-in files are read in name order, before the rest of `sshd_config`. A cloud image's `50-cloud-init.conf` can therefore override your setting if your file sorts later. Name your file with a low number, and always confirm the effective value with `sshd -T`. Two more traps: on Ubuntu 24.04+, a port change needs `systemctl daemon-reload` and `systemctl restart ssh.socket`; on RHEL with SELinux, a new SSH port also needs `semanage port -a -t ssh_port_t -p tcp PORT`.

Task 8: SSH logs

```
sudo journalctl -u ssh -n 50 --no-pager      # RHEL: -u sshd
sudo grep -E "Accepted|Failed" /var/log/auth.log | tail   # RHEL: /var/log/secure
```

Task 9: Host firewall

Action	Ubuntu (UFW)	RHEL-family (firewalld)
Status	`sudo ufw status verbose`	`sudo firewall-cmd --list-all`
Allow SSH	`sudo ufw allow OpenSSH`	`sudo firewall-cmd --permanent --add-service=ssh`
Allow HTTP	`sudo ufw allow 80/tcp`	`sudo firewall-cmd --permanent --add-service=http`
Activate	`sudo ufw enable`	`sudo firewall-cmd --reload`

> **WARNING**
>
> Allow SSH **before** you run `ufw enable` on a remote server. UFW's default policy denies incoming traffic, and your next SSH connection would be blocked.

Optional, needs a second machine: run `sudo ufw deny 80/tcp` and test from the other machine. The connection *times out*, because `deny` drops packets. Replace the rule with `sudo ufw reject 80/tcp` and test again: now it is *refused*. Delete the rule afterwards (`sudo ufw status numbered`, then `sudo ufw delete N`). Traffic from the server to itself over `lo` is always allowed, so you cannot see this effect locally.

Troubleshooting challenges

Case	First commands	Common causes
DNS does not resolve	`getent hosts`, `dig`, `resolvectl status`	Wrong resolver, typo in the record, blocked port 53
Port 80 not listening	`systemctl status nginx`, `ss -tlnp`	Service down, config error, port in use
SSH refused	`systemctl status ssh`, `ss -tlnp`	sshd stopped, wrong port
SSH times out	`nc -zv -w 3` from the client	Security group, NACL, UFW/firewalld, no public IP or route
Works locally, not remotely	`ss -tlnp` (bind address), firewall status	Bound to `127.0.0.1`, host firewall, security group

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- TCP vs UDP, a listening socket, and the bind addresses `0.0.0.0` vs `127.0.0.1`.
- The DNS resolution path: `nsswitch.conf`, `/etc/hosts`, the resolver, and why `dig` differs from `getent`.
- Connection refused vs connection timed out.
- Security group (stateful, instance level) vs NACL (stateless, subnet level) vs Linux firewall.

6 Bash Scripting and Automation Projects

Lab	Level	Revises	Best environment	Time
Lab 6	Intermediate–Advanced	Chapter 10 (cron and timers: Chapter 5)	VM or EC2 with Nginx from Lab 3	2 hours

LAB OBJECTIVE

Write scripts that are safe to rerun, return correct exit codes and run unattended from cron or a systemd timer.

Scenario: Your team manages several Linux servers. You need automated health checks, backups and service monitoring.

Every script below is created with a quoted here-document (`<<'EOF'`). The quotes stop the shell from expanding `$VARIABLES` while it writes the file. Install `shellcheck` (Ubuntu: `apt`; RHEL: EPEL package `ShellCheck`) and run it on every script you write.

Task 1: System information script

```
cat > ~/linux-labs/scripts/system-info.sh <<'EOF'
#!/bin/bash
echo "===== SYSTEM INFORMATION ====="
echo "Hostname: $(hostname)"
echo "Date:     $(date)"
echo "Uptime:   $(uptime -p)"
echo "===== MEMORY ====="
free -h
echo "===== DISK ====="
df -h -x tmpfs -x devtmpfs
echo "===== IP ADDRESS ====="
hostname -I
EOF
chmod +x ~/linux-labs/scripts/system-info.sh
~/linux-labs/scripts/system-info.sh
```

Task 2: Disk monitoring script

```
cat > ~/linux-labs/scripts/disk-monitor.sh <<'EOF'
#!/bin/bash
THRESHOLD=80
USAGE=$(df -P / | awk 'NR==2 {gsub("%", "", $5); print $5}')

if [ -z "$USAGE" ]; then
    echo "Unable to read disk usage" >&2
    exit 2
fi

if [ "$USAGE" -ge "$THRESHOLD" ]; then
    echo "WARNING: Root disk usage is ${USAGE}%"
    exit 1
fi
echo "Disk usage is normal: ${USAGE}%"
EOF
chmod +x ~/linux-labs/scripts/disk-monitor.sh
~/linux-labs/scripts/disk-monitor.sh; echo "exit=$?"
```

`df -P` forces one line per filesystem. Without it, a long device name can wrap onto two lines and break the `awk` field numbers.

Task 3: Service check with an argument

```
cat > ~/linux-labs/scripts/service-check.sh <<'EOF'
#!/bin/bash
SERVICE="${1:-nginx}"     # default to nginx if no argument is given

if systemctl is-active --quiet "$SERVICE"; then
    echo "$SERVICE is running"
else
    echo "$SERVICE is NOT running" >&2
    exit 1
fi
EOF
chmod +x ~/linux-labs/scripts/service-check.sh
~/linux-labs/scripts/service-check.sh nginx; echo "exit=$?"
~/linux-labs/scripts/service-check.sh no-such-service; echo "exit=$?"
```

Task 4: Backup script with verification and retention

```
cat > ~/linux-labs/scripts/backup.sh <<'EOF'
#!/bin/bash
set -Eeuo pipefail

SOURCE="$HOME/linux-labs/projects"
DEST="$HOME/linux-labs/backups"
KEEP_DAYS=7

[ -d "$SOURCE" ] || { echo "Source not found: $SOURCE" >&2; exit 1; }
mkdir -p "$DEST"

BACKUP="$DEST/projects-$(date +%F-%H%M%S).tar.gz"
tar -czf "$BACKUP" -C "$(dirname "$SOURCE")" "$(basename "$SOURCE")"

# Verify: the archive must be readable and not empty
tar -tzf "$BACKUP" > /dev/null
[ -s "$BACKUP" ] || { echo "Backup is empty" >&2; exit 1; }

# Retention: delete only OUR old archives, matched by name
find "$DEST" -maxdepth 1 -name 'projects-*.tar.gz' -mtime +"$KEEP_DAYS" -delete

echo "Backup created: $BACKUP ($(du -h "$BACKUP" | cut -f1))"
EOF
chmod +x ~/linux-labs/scripts/backup.sh
~/linux-labs/scripts/backup.sh
```

Now prove the backup can be restored. A backup you have never restored is not a backup.

```
LATEST=$(ls -t ~/linux-labs/backups/projects-*.tar.gz | head -1)
mkdir -p /tmp/restore-test && tar -xzf "$LATEST" -C /tmp/restore-test
diff -r ~/linux-labs/projects /tmp/restore-test/projects && echo "Restore OK"
rm -rf /tmp/restore-test
```

Task 5: Build a server health-check script

Requirements: check CPU load, memory, disk on every real filesystem and the required services. Log every result with a timestamp. Exit non-zero if any check fails. Try writing it yourself first, then compare with this reference solution:

```
cat > ~/linux-labs/scripts/health-check.sh <<'EOF'
#!/bin/bash
# No "set -e": a failing check must not stop the remaining checks.
set -uo pipefail

LOG="$HOME/linux-labs/logs/health-check.log"
DISK_LIMIT=80
MEM_LIMIT=90
SERVICES=("nginx")
FAILED=0

log() { echo "$(date '+%F %T') $*" | tee -a "$LOG"; }

# 1. CPU: 1-minute load average vs number of cores
CORES=$(nproc)
LOAD=$(awk '{print $1}' /proc/loadavg)
if awk -v l="$LOAD" -v c="$CORES" 'BEGIN { exit !(l > c) }'; then
    log "FAIL load=$LOAD cores=$CORES"; FAILED=1
else
    log "OK   load=$LOAD cores=$CORES"
fi

# 2. Memory: based on MemAvailable, not "free"
MEM_USED=$(awk '/^MemTotal/{t=$2} /^MemAvailable/{a=$2} END{printf "%d", (t-a)*100/t}' /p
if [ "$MEM_USED" -ge "$MEM_LIMIT" ]; then
    log "FAIL memory=${MEM_USED}%"; FAILED=1
else
    log "OK   memory=${MEM_USED}%"
fi

# 3. Disk: every real filesystem
while read -r USE MOUNT; do
    USE=${USE%\%}
    if [ "$USE" -ge "$DISK_LIMIT" ]; then
        log "FAIL disk $MOUNT=${USE}%"; FAILED=1
    else
        log "OK   disk $MOUNT=${USE}%"
    fi
done < <(df --output=pcent,target -x tmpfs -x devtmpfs -x squashfs -x overlay | tail -n +

# 4. Services
for SVC in "${SERVICES[@]}"; do
    if systemctl is-active --quiet "$SVC"; then
        log "OK   service $SVC"
    else
        log "FAIL service $SVC"; FAILED=1
    fi
done
```

```
exit "$FAILED"
EOF
chmod +x ~/linux-labs/scripts/health-check.sh
~/linux-labs/scripts/health-check.sh; echo "exit=$?"
sudo systemctl stop nginx
~/linux-labs/scripts/health-check.sh; echo "exit=$?"     # exit=1
sudo systemctl start nginx
```

Expected (healthy run):

```
2026-11-05 10:40:01 OK   load=0.12 cores=2
2026-11-05 10:40:01 OK   memory=31%
2026-11-05 10:40:01 OK   disk ╞═24%
2026-11-05 10:40:01 OK   service nginx
exit=0
```

Task 6: Schedule with cron

Item	Ubuntu	RHEL-family
Service	`cron`	`crond` (package `cronie`)
Cron log	`journalctl -u cron` or `grep CRON /var/log/syslog`	`/var/log/cron` or `journalctl -u crond`
Amazon Linux 2023	–	cron is **not installed**: `sudo dnf install -y cronie && sudo systemctl enable --now crond`, or use a systemd timer

```
crontab -e
```

```
*/2 * * * * /home/USERNAME/linux-labs/scripts/health-check.sh >> /home/USERNAME/linux-lab
0 2 * * *   /usr/bin/flock -n /tmp/backup.lock /home/USERNAME/linux-labs/scripts/backup.s
```

Wait a few minutes, then check `crontab -l`, the log files and the cron log. `flock -n` skips a run if the previous one is still going, so long jobs never overlap.

> **INTERVIEW TRAP**
>
> In a crontab line, `%` means "newline". `tar -czf /backup/etc-$(date +%F).tar.gz /etc` works in a shell but breaks in cron. Escape it as `\%`, or move the command into a script.

Task 7: The same job as a systemd timer

```
sudo tee /etc/systemd/system/health-check.service >/dev/null <<'EOF'
[Unit]
Description=Lab health check

[Service]
Type=oneshot
User=USERNAME
ExecStart=/home/USERNAME/linux-labs/scripts/health-check.sh
EOF

sudo tee /etc/systemd/system/health-check.timer >/dev/null <<'EOF'
[Unit]
Description=Run the lab health check every 5 minutes

[Timer]
OnCalendar=*:0/5
Persistent=true

[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now health-check.timer
systemctl list-timers health-check.timer
journalctl -u health-check.service -n 20 --no-pager
```

Replace `USERNAME` in both units. When the script exits 1, the service shows as `failed` in `systemctl --failed`. Monitoring tools can alert on that state. Disable either the cron entry or the timer so the check does not run twice.

Troubleshooting challenges

Challenge 1 **Works manually, fails in cron**

Capture cron's environment, then compare it with your shell's:

```
* * * * * env > /tmp/cron-env.txt          # add to crontab, wait one minute, then re
diff <(sort /tmp/cron-env.txt) <(env | sort)
```

Cron runs with a minimal `PATH`, no profile files and a different working directory. Fix this with absolute paths, a `PATH=` line at the top of the crontab, and logging of stderr (`2>&1`).

Challenge 2 Paths with spaces

```
DIR="$HOME/linux-labs/my projects"
mkdir -p "$DIR"
ls $DIR        # fails: word splitting turns it into two arguments
ls "$DIR"      # works
```

Find every unquoted variable in your scripts. `shellcheck` flags them all (SC2086).

Challenge 3 Backup archive is empty or missing

Temporarily change `SOURCE` to a path that does not exist and run the backup. Your script should exit non-zero with a clear message. A silent "success" is a failure. Then restore the correct path.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Variables, quoting, conditions, loops, functions and exit codes.
- What `set -Eeuo pipefail` does, and when `set -e` is the wrong choice.
- Cron syntax, cron's environment, and cron vs systemd timers.
- Why scripts must be **idempotent** (safe to rerun) and must fail loudly.

7 Log Analysis and Monitoring

Lab	Level	Revises	Best environment	Time
Lab 7	Intermediate	Chapters 11–12	VM or EC2 with Nginx from Lab 3	60 min

LAB OBJECTIVE

Find the right log quickly, filter the journal, summarise access logs with `awk`, `sort` and `uniq`, and understand log rotation.

Scenario: Users report intermittent errors. You must find patterns in the logs and report them with evidence.

Task 1: Generate realistic traffic

```
for i in $(seq 1 30); do curl -s -o /dev/null http://localhost/; done
for i in $(seq 1 8);  do curl -s -o /dev/null http://localhost/missing-page; done
curl -s -o /dev/null -X POST http://localhost/      # 405 Not Allowed on a static site
```

Task 2: Know where the logs live

Log	Ubuntu	RHEL-family
General system	`/var/log/syslog`	`/var/log/messages`
Authentication	`/var/log/auth.log`	`/var/log/secure`
Cron	Inside `syslog`	`/var/log/cron`
Kernel	`/var/log/kern.log`	Inside `messages`
Nginx	`/var/log/nginx/access.log` , `error.log`	Same paths

Amazon Linux 2023 does not install rsyslog by default. Most system logs exist only in the journal, so use `journalctl`. On Ubuntu, Nginx logs belong to `www-data:adm` with mode `640`. Read them with `sudo`, or add your user to the `adm` group.

```
ls -lah /var/log
sudo ls -l /var/log/nginx/
```

Task 3: Filter the systemd journal

```
sudo journalctl -n 100 --no-pager
sudo journalctl -u nginx --since "1 hour ago" --no-pager
sudo journalctl -p err -b --no-pager        # errors and worse, current boot
sudo journalctl -u ssh -f                    # follow live (RHEL: -u sshd)
journalctl --disk-usage
```

Task 4: Analyse the Nginx access log

The default combined log format has these fields:

```
$1 client IP   $4 [time   $6 "METHOD   $7 path   $8 protocol"   $9 status   $10 bytes
127.0.0.1 - - [05/Nov/2026:10:17:01 +0000] "GET /missing-page HTTP/1.1" 404 162 "-" "curl
```

```
# Status codes, most frequent first
sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -nr

# Most requested paths
sudo awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head

# Top client IPs
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head

# Every 4xx/5xx with time and path
sudo awk '$9 ~ /^[45]/ {print $4, $7, $9}' /var/log/nginx/access.log

# Errors per minute (spot a spike)
sudo awk '$9 ~ /^[45]/ {print substr($4, 2, 17)}' /var/log/nginx/access.log | sort | uniq
```

Expected (status summary):

```
     31 200
      8 404
      1 405
```

Status	Meaning	Where to look first
404	Path not found	Wrong URL, wrong `root`, missing deployment files
500	Application error	Application logs
502	Bad gateway: upstream down or refusing	Upstream service status, port, Nginx `error.log`
503	Service unavailable	Overload, rate limits, maintenance mode
504	Gateway timeout: upstream too slow	Slow application or database, proxy timeouts

Task 5: Watch logs live

```
sudo tail -f /var/log/nginx/access.log
# In a second terminal:
curl -s http://localhost/ > /dev/null; curl -s http://localhost/nope > /dev/null
```

Task 6: Build and summarise an application log

```
cd ~/linux-labs/logs
for i in $(seq 1 200); do
    case $((RANDOM % 10)) in
        0)   LEVEL=ERROR ;;
        1|2) LEVEL=WARNING ;;
        *)   LEVEL=INFO ;;
    esac
    echo "$(date '+%F %T') $LEVEL [module$((RANDOM % 3))] request id=$i"
done > app-test.log

awk '{print $3}' app-test.log | sort | uniq -c | sort -nr            # count per level
awk '$3=="ERROR" {print $4}' app-test.log | sort | uniq -c | sort -nr  # errors per modul
```

Task 7: Log rotation

```
cat /etc/logrotate.d/nginx
sudo logrotate -d /etc/logrotate.d/nginx     # dry run: shows what WOULD happen
ls -l /var/log/nginx/
```

The `postrotate` block tells Nginx to reopen its log files. Without it, Nginx keeps writing to the rotated (renamed or deleted) file, and the disk fills while the new log stays empty. `copytruncate` is the alternative for applications that cannot reopen logs, but it can lose a few lines during the copy.

Task 8: Resources and kernel messages

```
top                       # q to quit
free -h
vmstat 1 5
iostat -xz 1 5            # from sysstat
sudo journalctl -k -n 100 --no-pager
sudo dmesg -T | tail -n 50   # Ubuntu restricts dmesg for normal users
```

`sar` history needs data collection enabled. On Ubuntu, set `ENABLED="true"` in `/etc/default/sysstat`, then run `sudo systemctl enable --now sysstat`. On RHEL-family systems, `sudo systemctl enable --now sysstat` is enough.

Troubleshooting challenges

Challenge 1 The manager's question

"How many 404s did we serve, for which paths, and from which IPs?" Answer with one pipeline per question. Paste the commands and output into a short report.

Challenge 2 **Level summary report**

From `app-test.log`, produce a report that shows the total line count, the count per level, the error percentage and the module with the most errors. Save it as `~/linux-labs/logs/report.txt`. Bonus: turn it into a script that takes the log file as an argument.

INTERVIEW TRAP

"The access log is empty but users say the site works." Check whether logrotate rotated the file without telling Nginx to reopen it (`lsof -p NGINX_PID | grep log` shows the deleted file). Also check whether the site's server block sends its logs to a different `access_log` path.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Access log vs error log, and journald vs rsyslog files.
- How to find 404, 500, 502 and 504 patterns, and what each code points to.
- How log rotation works and why services must reopen their logs.
- Which tool answers which question: `top`, `vmstat`, `iostat`, `journalctl -k`.

8 CPU, Memory and Performance Troubleshooting

Lab	Level	Revises	Best environment	Time
Lab 8	Advanced	Chapter 13 (ulimit: Chapter 5)	Dedicated test VM or EC2	75 min

LAB OBJECTIVE

Create controlled CPU, memory and I/O pressure. Then find the bottleneck with standard tools, the same way you would during an incident.

Scenario: An application server becomes slow during peak traffic. You must find out whether CPU, memory or disk is the bottleneck, and prove it.

WARNING

Generate load only on your own test machine, and always with a `--timeout`. Never run these exercises on shared, production or burstable instances that other people depend on.

```
sudo apt install -y stress-ng        # RHEL-family: sudo dnf install -y stress-ng (AppStr
```

If `stress-ng` is not available (for example on Amazon Linux 2023), run `yes > /dev/null &` once per core to load the CPU, and `pkill yes` to stop it.

Task 1: Load average vs cores

```
uptime
nproc
cat /proc/loadavg
```

Load average counts tasks that are running, waiting for a CPU, **or stuck in uninterruptible I/O wait (D state)**. A load of 4.0 on 4 cores is full. A load of 4.0 on 16 cores is quiet.

Task 2: CPU bottleneck

```
stress-ng --cpu "$(nproc)" --timeout 120s &
# In a second terminal while it runs:
uptime
top                          # press 1 (per-CPU), P (sort by CPU), q (quit)
ps aux --sort=-%cpu | head -5
mpstat -P ALL 1 3
pidstat -u 1 3
```

Task 3: Memory pressure

```
free -h
stress-ng --vm 1 --vm-bytes 60% --vm-keep --timeout 60s &
free -h
vmstat 1 5
ps aux --sort=-%mem | head -5
ps -o pid,rss,vsz,comm -p "$(pgrep -d, stress-ng)"
```

Expected `free -h` before the test:

```
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.1Gi       0.4Gi        12Mi       2.5Gi       2.7Gi
Swap:             0B          0B          0B
```

"free" looks low, but **available** (2.7 GiB) is the real number. The kernel uses spare memory as page cache (`buff/cache`) and gives it back when applications need it. **RSS** is physical memory in use; **VSZ** is reserved virtual address space, which is often much larger and is not a problem by itself.

```
grep -E 'MemTotal|MemAvailable|^Cached|SwapTotal|SwapFree|Dirty' /proc/meminfo
```

Task 4: Read vmstat

Column	Meaning	Worry when
`r`	Runnable tasks	Consistently above the core count
`b`	Tasks blocked on I/O (D state)	Consistently above zero
`si` / `so`	Swap in / out per second	Non-zero and sustained (memory pressure)
`us` / `sy`	User / kernel CPU %	`sy` high: syscall-heavy or kernel work
`wa`	CPU idle while waiting for I/O	High: storage is the bottleneck
`st`	Steal: CPU taken by the hypervisor	High on cloud VMs: noisy neighbour or burst credits used up

Task 5: Disk I/O bottleneck

```
stress-ng --hdd 1 --hdd-bytes 512M --temp-path ~/linux-labs --timeout 60s &
iostat -xz 1 5            # watch %util, r_await / w_await, aqu-sz
pidstat -d 1 3            # I/O per process
ps -eo state,pid,cmd | awk '$1=="D"'   # processes in uninterruptible sleep
```

DEVOPS NOTE

On AWS, compare what you see with the volume's limits. A gp3 volume has a baseline of 3,000 IOPS and 125 MiB/s unless you provision more. In CloudWatch, watch `VolumeQueueLength` for EBS, `BurstBalance` for gp2, and `CPUCreditBalance` for T-family instances. An empty credit balance shows up as high `st` inside Linux.

Task 6: A controlled OOM kill

Run the memory hog inside a systemd scope capped at 100 MB, so only that scope can be killed:

```
sudo systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 \
    stress-ng --vm 1 --vm-bytes 300M --timeout 20s
sudo journalctl -k --since "5 min ago" --no-pager | grep -i -E 'oom|killed process'
```

You should see a `Memory cgroup out of memory` message. This is the same mechanism that kills containers that exceed their memory limit (Lab 9).

Task 7: File descriptor limits

```
ulimit -n; ulimit -Hn                                    # your shell's soft / hard lim
NGINX_PID=$(systemctl show -p MainPID --value nginx)
sudo grep "open files" /proc/$NGINX_PID/limits
sudo ls /proc/$NGINX_PID/fd | wc -l                      # descriptors in use
systemctl show nginx -p LimitNOFILE
```

To change the limit for a service, use a drop-in, not `limits.conf`:

```
sudo systemctl edit nginx
# add:  [Service]
#       LimitNOFILE=65535
sudo systemctl restart nginx
sudo grep "open files" /proc/$(systemctl show -p MainPID --value nginx)/limits
```

INTERVIEW TRAP

"You raised nofile in `/etc/security/limits.conf` but the service still hits 'Too many open files'. Why?" `limits.conf` is applied by PAM at login. systemd services do not log in through PAM, so they ignore it. Set `LimitNOFILE=` in the unit or a drop-in, restart the service, and verify in `/proc/PID/limits`.

Troubleshooting challenge

Challenge 1 The mystery slow server

Ask a partner to start one hidden `stress-ng` workload (CPU, VM or HDD) with a 10-minute timeout. Do not look at their terminal. In under 10 minutes, find the bottleneck and the process causing it, then write an incident report using this template:

```
INCIDENT SUMMARY
Time detected   :
Symptom         : (what users or alerts saw)
Impact          :
Evidence        : (commands run + key output lines)
Suspected cause :
Action taken    :
Follow-up       : (alert, limit, capacity change)
```

INTERVIEW TRAP QUESTION

Load average is 12 on an 8-core server, but top shows the CPU 70% idle. What is happening?

The load is coming from tasks in D state, which are waiting on I/O, not on CPU. Check `vmstat` (`b` and `wa` columns), `iostat -xz` and `ps` for D-state processes. Common causes are a saturated disk, an EBS volume at its IOPS limit, or a hung NFS/EFS mount.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Load average, and CPU-bound vs I/O-bound slowness.
- free vs available memory, page cache, swap activity, RSS vs VSZ.
- How the OOM killer chooses and logs its victim, and how to find OOM events.
- File descriptor limits for shells vs systemd services.

9 Docker and Kubernetes Linux Labs

Lab	Level	Revises	Best environment	Time
Lab 9	Advanced	Chapters 14–15	VM or EC2 with 4 GB RAM	2 hours

LAB OBJECTIVE

See with your own eyes that a container is just a Linux process with namespaces, cgroups and an overlay filesystem. Then practise the most common container and pod failures.

Scenario: Your organisation is moving an application from a traditional server to Docker and Kubernetes. You must be able to debug it at the Linux level.

Task 1: Install a container engine

System	Install
Ubuntu	Docker CE from Docker's official repository (docs.docker.com), or `sudo apt install -y docker.io` for labs
RHEL / Rocky / Alma	Docker CE repository, or Podman: `sudo dnf install -y podman podman-docker` (most commands are the same)
Amazon Linux 2023	`sudo dnf install -y docker && sudo systemctl enable --now docker`

```
docker --version
sudo docker info | grep -Ei 'storage driver|cgroup'
```

> **WARNING**
>
> Adding a user to the `docker` group (`sudo usermod -aG docker $USER`) is equivalent to giving that user root. Anyone in the group can run `docker run -v /:/host`. Do it only on lab machines. The examples below use `docker` without `sudo`, assuming you joined the group and logged in again.

Task 2: Run and test a container

```
docker run -d --name web-lab -p 8080:80 nginx
docker ps
curl -I http://localhost:8080
```

Task 3: A container is a host process

```
docker top web-lab                                  # processes with HOST PIDs
PID=$(docker inspect -f '{{.State.Pid}}' web-lab)
ps -p "$PID" -o pid,user,cmd                       # visible on the host
docker exec web-lab cat /proc/1/comm               # inside: nginx is PID 1
```

The official `nginx` image has no `ps` command, which is why we read `/proc` directly. Inside the container the master process is PID 1. On the host, the same process has an ordinary PID.

Task 4: Logs, and where they live

```
docker logs --tail 20 web-lab
docker inspect -f '{{.LogPath}}' web-lab            # JSON log file on the host
docker stats --no-stream
```

INTERVIEW TRAP

The default `json-file` log driver does **not** rotate logs. A chatty container can fill `/var/lib/docker` and take down the host. Set `"log-opts": {"max-size": "10m", "max-file": "3"}` in `/etc/docker/daemon.json`. The setting applies to containers created after the change.

Task 5: Volumes and bind-mount permissions

```
docker volume create web-data
docker run -d --name volume-lab -v web-data:/data nginx
docker exec volume-lab sh -c 'echo hello > /data/test.txt'
docker rm -f volume-lab
docker run --rm -v web-data:/data alpine cat /data/test.txt   # data survived
docker volume inspect web-data                                # Mountpoint on the host

# Bind mount as a non-root container user
sudo mkdir ~/linux-labs/root-owned
docker run --rm --user 1000:1000 -v ~/linux-labs/root-owned:/data alpine touch /data/x
# touch: /data/x: Permission denied - UIDs are the same numbers inside and outside
```

Fix it by matching ownership to the container's UID: `sudo chown 1000:1000 ~/linux-labs/root-owned`. On RHEL with SELinux, bind mounts also need a label: add `:Z` (private) or `:z` (shared) to the `-v` option.

Task 6: Namespaces and cgroups by hand

```
PID=$(docker inspect -f '{{.State.Pid}}' web-lab)
sudo ls -l /proc/$PID/ns
sudo readlink /proc/1/ns/net /proc/$PID/ns/net      # different IDs = separate network na
sudo nsenter -t "$PID" -n ip -br addr               # host tool, container network
stat -fc %T /sys/fs/cgroup                          # cgroup2fs = cgroup v2
sudo cat /proc/$PID/cgroup
```

Expected (IDs will differ):

```
net:[4026531840]
net:[4026532301]
```

Now set a memory limit and find it in the cgroup filesystem:

```
docker run -d --name mem-lab --memory 128m nginx
MPID=$(docker inspect -f '{{.State.Pid}}' mem-lab)
CG=$(sudo awk -F: '{print $3}' /proc/$MPID/cgroup)
cat /sys/fs/cgroup${CG}/memory.max                  # 134217728 = 128 MiB
findmnt -t overlay | head -3                        # the container's layered root filesy
```

Feature	What it isolates or limits	How you saw it
PID namespace	Process IDs (own PID 1)	`/proc/1/comm` inside vs host PID
Network namespace	Interfaces, routes, ports	`readlink .../ns/net`, `nsenter -n`
Mount namespace	Filesystem view	Overlay mounts in `findmnt`
User namespace	UID mapping (rootless containers)	`/proc/PID/uid_map`
cgroups	CPU, memory and I/O limits and accounting	`memory.max`
OverlayFS	Read-only image layers + a writable layer	`findmnt -t overlay`

Task 7: Reproduce container failures

```
# 1. Exits immediately: no long-running foreground process
docker run -d --name exit-lab alpine
docker ps -a --filter name=exit-lab
docker inspect -f '{{.State.ExitCode}}' exit-lab

# 2. OOMKilled: a 32 MB limit and a string that keeps doubling
docker run --name oom-lab --memory 32m --memory-swap 32m alpine \
    sh -c 'x=a; while true; do x=$x$x; done'
docker inspect -f '{{.State.OOMKilled}} {{.State.ExitCode}}' oom-lab   # true 137
```

Exit code **137** means 128 + 9: the process was killed by SIGKILL. Combined with `OOMKilled=true`, that SIGKILL came from the memory limit.

Task 8: Kubernetes on a test cluster

Use a local test cluster: kind, minikube or k3s. Port 8080 is still used by `web-lab`, so forward to 8082:

```
kubectl create deployment nginx-lab --image=nginx
kubectl get deployments,pods -o wide
kubectl port-forward deployment/nginx-lab 8082:80     # leave running
curl -I http://localhost:8082                          # second terminal
kubectl logs deployment/nginx-lab
kubectl get events --sort-by=.metadata.creationTimestamp
```

Now reproduce the classic pod failures:

```
# CrashLoopBackOff
kubectl run crash-lab --image=busybox --restart=Always -- sh -c "echo starting; exit 1"
kubectl get pod crash-lab -w            # Error, then CrashLoopBackOff (Ctrl+C to stop wa
kubectl logs crash-lab --previous       # logs of the LAST crashed container

# ImagePullBackOff
kubectl run pull-lab --image=nginx:no-such-tag
kubectl describe pod pull-lab | tail -n 15
```

For an OOMKilled pod, `kubectl describe pod` shows `Last State: Terminated`, `Reason: OOMKilled` and `Exit Code: 137`. It is the same cgroup mechanism you just saw in Docker.

Troubleshooting challenges

Symptom	First commands	Typical cause
Container exits immediately	`docker ps -a`, `docker logs`, exit code	No foreground process, bad command, missing config
Port mapping does not work	`docker ps` (PORTS), `ss -tlnp`, `curl` inside	App bound to 127.0.0.1 inside, wrong container port, host port taken
Cannot write to mounted directory	`ls -ln` on host, `id` in container	UID mismatch, SELinux label missing (`:Z`)
CrashLoopBackOff	`kubectl logs --previous`, `describe`	App crash, bad env or config, failing liveness probe
OOMKilled	`describe pod`, exit code 137	Memory limit too low, or a memory leak
ImagePullBackOff	`describe pod` (Events)	Wrong tag, private registry without credentials, no egress

WARNING

Clean up:

```
docker rm -f web-lab mem-lab exit-lab oom-lab
docker volume rm web-data
sudo rm -rf ~/linux-labs/root-owned
kubectl delete deployment nginx-lab
kubectl delete pod crash-lab pull-lab
```

INTERVIEW TRAP QUESTION

A container hits its CPU limit. Does Kubernetes kill it, like it does when it hits the memory limit?

No. CPU is a compressible resource: the container is **throttled**, so it runs slower but keeps running. Memory is not compressible: exceeding the limit triggers the OOM killer, and you see `OOMKilled` with exit code 137.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Containers vs virtual machines (containers share the host kernel).
- Namespaces vs cgroups, and what OverlayFS adds.
- Container runtime, volumes vs bind mounts, and log rotation for containers.
- How you debug CrashLoopBackOff, ImagePullBackOff and OOMKilled pods.

10 AWS EC2 Linux Administration

Lab	Level	Revises	Best environment	Time
Lab 10	Advanced	Chapter 17 (EBS: Chapter 6; VPC: Chapter 7)	Test AWS account	2 hours

LAB OBJECTIVE

Launch and administer Linux EC2 instances: SSH, user data, IMDSv2, Session Manager, EBS volumes and IAM roles. Practise the AWS-specific failure modes.

Scenario: You have been given an EC2 instance to host an application. You must set it up securely, add storage and access S3 without any stored keys.

WARNING

This lab costs money. Check current EC2 pricing and your Free Tier terms before you start. Instances, EBS volumes (even detached ones), snapshots, Elastic IPs and public IPv4 addresses are all billed. The cleanup step at the end of this lab is not optional.

Task 1: Launch the instance

- **AMI:** Ubuntu Server LTS, or Amazon Linux 2023 for the RHEL-family path.
- **Instance type:** a small burstable type (t3 or t4g family). t4g is ARM, and every command here works the same.
- **Key pair:** a new ED25519 key pair just for this lab.
- **Network:** a public subnet whose route table sends `0.0.0.0/0` to an internet gateway, with "auto-assign public IP" enabled.
- **Security group:** SSH (22) from **My IP** only; HTTP (80) from anywhere.
- **Advanced details:** metadata version **V2 only (token required)**.
- **IAM instance profile:** a role named `lab-ec2-role` with the managed policy `AmazonSSMManagedInstanceCore`. You will add S3 access in Task 8.

Task 2: Connect with SSH

```
chmod 400 my-key.pem
ssh -i my-key.pem ubuntu@PUBLIC_IP
```

AMI	Default username
Ubuntu	`ubuntu`
Amazon Linux 2023	`ec2-user`
RHEL	`ec2-user`
Debian	`admin`

A wrong username gives `Permission denied (publickey)`. It looks like a key problem but is not.

Task 3: Inspect the server, the cloud way

```
cat /etc/os-release
cloud-init status --long
lsblk                    # Nitro instances show NVMe devices: nvme0n1
ip -br addr              # only the PRIVATE IP is on the interface
df -h; free -h
```

INTERVIEW TRAP

"Why can't you see the public IP in `ip addr`?" The public IPv4 address is never configured inside the instance. The internet gateway translates (NATs) it to the private IP. To find it from inside, query instance metadata (Task 5).

Task 4: Install Nginx and test from outside

Step	Ubuntu	Amazon Linux 2023
Install	`sudo apt update && sudo apt install -y nginx`	`sudo dnf install -y nginx`
Start	Automatic	`sudo systemctl enable --now nginx`

Open `http://PUBLIC_IP` in your browser. If it times out, check the security group first. Amazon Linux 2023 has no host firewall by default.

Task 5: User data and IMDSv2

Launch a second test instance with this user data:

```
#!/bin/bash
set -euo pipefail
# Ubuntu
apt-get update -y
apt-get install -y nginx
systemctl enable --now nginx
# Amazon Linux 2023 instead: dnf install -y nginx && systemctl enable --now nginx
```

```
cloud-init status --wait
sudo tail -n 50 /var/log/cloud-init-output.log
sudo cat /var/lib/cloud/instance/user-data.txt
```

User data runs as root, once, on the first boot. Changing it on a stopped instance does not run it again by default.

Now query metadata with IMDSv2:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
MD="http://169.254.169.254/latest/meta-data"
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/instance-id; echo
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/instance-type; echo
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/placement/availability-zone; echo
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/public-ipv4; echo
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/iam/security-credentials/; echo   # rol

# Without a token (IMDSv1) the request is refused:
curl -s -o /dev/null -w '%{http_code}\n' $MD/     # 401
```

PRODUCTION WARNING

Never put passwords, tokens or keys in user data. Any process on the instance can read user data through the metadata service. IMDSv2 protects against SSRF attacks. If code inside Docker containers on the instance cannot reach IMDSv2, check the metadata hop limit (containers usually need 2).

Task 6: Connect with Session Manager (no SSH)

Requirement	How to check
IAM role with `AmazonSSMManagedInstanceCore`	EC2 console → instance → Security tab
SSM Agent running	Ubuntu: `sudo snap services amazon-ssm-agent`; AL2023: `systemctl status amazon-ssm-agent`
Outbound HTTPS (443) to Systems Manager endpoints	Internet or NAT route, or VPC interface endpoints (`ssm` , `ssmmessages` , and `ec2messages` for older agents)
Agent log	`/var/log/amazon/ssm/amazon-ssm-agent.log`

```
# From your laptop (requires the Session Manager plugin):
aws ssm describe-instance-information --query 'InstanceInformationList[].[InstanceId,Ping
aws ssm start-session --target i-0123456789abcdef0
```

Now remove the SSH rule from the security group and connect again with Session Manager. It still works, because the agent makes the connection outbound. This is why many teams close port 22 completely.

Task 7: Add, mount and grow an EBS volume

1. Create a 1 GiB gp3 volume in the **same Availability Zone** as the instance, then attach it.
2. Identify it. On Nitro instances, a volume attached as `/dev/sdf` appears as `/dev/nvme1n1` .

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,SERIAL     # SERIAL shows the volume ID
sudo file -s /dev/nvme1n1                            # "data" = no filesystem
```

Expected:

```
NAME          SIZE TYPE FSTYPE MOUNTPOINT SERIAL
nvme0n1         8G disk                   vol0a1b2c3d4e5f6a7b8c
└─nvme0n1p1   7.9G part ext4   /
nvme1n1         1G disk                   vol0123456789abcdef0
```

WARNING

Format only a new, empty volume. Match the volume ID in `SERIAL` with the console first. Running `mkfs` on a volume restored from a snapshot destroys the data you restored.

```
sudo mkfs.ext4 -L appdata /dev/nvme1n1          # AL2023: sudo mkfs.xfs -L appdata /dev/n
sudo mkdir -p /data
sudo cp /etc/fstab /etc/fstab.bak-lab10
UUID=$(sudo blkid -s UUID -o value /dev/nvme1n1)
echo "UUID=$UUID /data ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab   # xfs on AL20
sudo mount -a && sudo findmnt --verify && df -h /data
sudo reboot                                      # reconnect, then: findmnt /data
```

Now grow the volume while it stays online:

```
aws ec2 modify-volume --volume-id vol-0123456789abcdef0 --size 2
aws ec2 describe-volumes-modifications --volume-ids vol-0123456789abcdef0
lsblk /dev/nvme1n1                # shows 2G once the modification is "optimizing" or "c
sudo resize2fs /dev/nvme1n1       # ext4 (whole disk, no partition)
# XFS: sudo xfs_growfs /data
# Partitioned disk (like the root volume): sudo growpart /dev/nvme0n1 1 first
df -h /data
```

AWS limits how often you can modify the same volume, so plan size changes instead of making many small ones.

Task 8: AWS CLI with an IAM role

The AWS CLI v2 is preinstalled on Amazon Linux 2023. On Ubuntu, use AWS's official installer or `sudo snap install aws-cli --classic`.

```
aws sts get-caller-identity
aws configure list              # the "Type" column should show iam-role
```

Expected:

```
{
    "UserId": "AROAEXAMPLEID:i-0123456789abcdef0",
    "Account": "123456789012",
    "Arn": "arn:aws:sts::123456789012:assumed-role/lab-ec2-role/i-0123456789abcdef0"
}
```

Create a test bucket, then add this least-privilege inline policy to `lab-ec2-role`, replacing `BUCKET`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::BUCKET" },
    { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::BUCKET/backups/*" }
  ]
}
```

```
tar -czf /tmp/etc-backup.tar.gz -C / etc 2>/dev/null
aws s3 cp /tmp/etc-backup.tar.gz s3://BUCKET/backups/
aws s3 ls s3://BUCKET/backups/     # works
aws s3 ls                          # AccessDenied: needs s3:ListAllMyBuckets, which we di
```

INTERVIEW TRAP

"The role has the right policy but the CLI still gets AccessDenied." Check `aws configure list`. Environment variables and `~/.aws/credentials` take precedence over the instance role, so an old access key left on the server silently wins. Also check the bucket policy, KMS key permissions and the exact resource ARN (`bucket` vs `bucket/*`).

Troubleshooting challenges

Symptom	Check, in order
SSH times out	Public IP, security group (port 22 from your current IP), NACL, route to the internet gateway, then sshd and host firewall
SSM instance not online	Instance profile and policy, agent status and log, outbound 443 or VPC endpoints
EBS volume not visible	Attached? Same AZ? Look for `nvme1n1` , not `xvdf` ; check `lsblk` and `dmesg`
Mount missing after reboot	`/etc/fstab` entry, UUID, `nofail` ; if boot fails, use the EC2 serial console or attach the root volume to a rescue instance
Website inaccessible externally	Security group port 80, `ss -tlnp` bind address, host firewall, curl from inside first
AccessDenied on S3	`aws sts get-caller-identity` , credential source, policy actions and ARNs, bucket policy, KMS

WARNING

Clean up everything you created: terminate both instances, delete the detached EBS volume and any snapshots, empty and delete the S3 bucket, release any Elastic IP, and remove the test key pair and security group. Check the Billing console the next day.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- EC2 networking: subnet, route table, internet gateway, security group vs NACL.
- EBS vs EFS vs instance store, and NVMe device naming on Nitro instances.
- User data, cloud-init, and why IMDSv2 matters.
- IAM roles vs access keys, and how Session Manager works without inbound ports.

11 Nginx, Apache, PHP and Node.js Deployment

Lab	Level	Revises	Best environment	Time
Lab 11	Advanced	Chapters 5, 9 and 16	Ubuntu VM or EC2 (RHEL notes included)	2.5 hours

LAB OBJECTIVE

Deploy a static site, a Node.js service behind an Nginx reverse proxy, a PHP-FPM site and Apache on one server. Then reproduce the classic web failures: 502s, source code leaks and port conflicts.

Scenario: You are responsible for deploying several applications on one Linux server.

Task 1: Know the Nginx layout

Item	Ubuntu	RHEL-family
Site files	`/etc/nginx/sites-available/NAME` + symlink in `sites-enabled/`	`/etc/nginx/conf.d/NAME.conf`
Worker user	`www-data`	`nginx`
SELinux	Not used (AppArmor instead)	Enforcing on RHEL / Rocky / Alma

Task 2: Static website

```
sudo mkdir -p /var/www/static-site
echo "<h1>Linux DevOps Deployment Lab</h1>" | sudo tee /var/www/static-site/index.html

sudo tee /etc/nginx/sites-available/static-lab >/dev/null <<'EOF'
server {
    listen 80;
    server_name static-lab.local;
    root /var/www/static-site;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF
sudo ln -s /etc/nginx/sites-available/static-lab /etc/nginx/sites-enabled/static-lab
# RHEL: write the same block to /etc/nginx/conf.d/static-lab.conf (no symlink)
sudo nginx -t && sudo systemctl reload nginx
curl -H "Host: static-lab.local" http://127.0.0.1
```

INTERVIEW TRAP

The here-document uses `<<'EOF'` with quotes. Without the quotes, the shell replaces `$uri` with an empty string while writing the file, and the config silently breaks. On RHEL, files created in your home directory and then moved with `mv` keep the wrong SELinux label, and Nginx returns 403. Fix the label with `sudo restorecon -Rv /var/www/static-site`.

Task 3: A Node.js application

Use a current Node.js LTS release: NodeSource packages, the official binaries from nodejs.org, or your distribution's module streams. On Amazon Linux 2023, run `dnf list available 'nodejs*'` to see the options. The Ubuntu archive version can be old. Avoid `nvm` for services, because it installs Node per user under a home directory.

```
command -v node && node --version && npm --version
mkdir -p ~/linux-labs/projects/node-app && cd ~/linux-labs/projects/node-app

cat > server.js <<'EOF'
const http = require("http");
const port = process.env.PORT || 3000;

http.createServer((req, res) => {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Node.js application running on Linux\n");
}).listen(port, "127.0.0.1", () => {
    console.log(`Server listening on 127.0.0.1:${port}`);
});
EOF
node server.js                          # test in a second terminal, then Ctrl+C
curl http://127.0.0.1:3000
```

Task 4: Run Node.js as a systemd service

Production services run from a fixed path as a dedicated non-login user, not from someone's home directory:

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin nodeapp
sudo mkdir -p /opt/node-lab
sudo cp ~/linux-labs/projects/node-app/server.js /opt/node-lab/
sudo chown -R root:nodeapp /opt/node-lab
sudo chmod 750 /opt/node-lab && sudo chmod 640 /opt/node-lab/server.js

sudo tee /etc/systemd/system/node-lab.service >/dev/null <<'EOF'
[Unit]
Description=Node.js Lab Application
After=network.target

[Service]
Type=simple
User=nodeapp
Group=nodeapp
WorkingDirectory=/opt/node-lab
Environment=NODE_ENV=production PORT=3000
ExecStart=/usr/bin/node /opt/node-lab/server.js
Restart=on-failure
RestartSec=5
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now node-lab
systemctl status node-lab --no-pager
sudo ss -tlnp | grep 3000              # 127.0.0.1:3000
```

`ExecStart` must use the real path from `command -v node`. The official tarball installs to `/usr/local/bin/node`. A wrong path fails with:

```
node-lab.service: Main process exited, code=exited, status=203/EXEC
```

Test automatic restarts:

```
sudo kill -9 "$(systemctl show -p MainPID --value node-lab)"
sleep 6; systemctl status node-lab --no-pager      # restarted, with a new PID
```

INTERVIEW TRAP

`Restart=on-failure` restarts after SIGKILL or a non-zero exit. It does **not** restart after a "clean" signal such as SIGTERM. If someone runs `kill PID` (SIGTERM by default), the service stays down. Use `Restart=always` when the service must come back regardless of how it stopped.

Task 5: Nginx as a reverse proxy

```
sudo tee /etc/nginx/sites-available/node-lab >/dev/null <<'EOF'
server {
    listen 80;
    server_name node-lab.local;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
sudo ln -s /etc/nginx/sites-available/node-lab /etc/nginx/sites-enabled/node-lab
sudo nginx -t && sudo systemctl reload nginx
curl -H "Host: node-lab.local" http://127.0.0.1
```

Now create a 502 on purpose:

```
sudo systemctl stop node-lab
curl -I -H "Host: node-lab.local" http://127.0.0.1     # HTTP/1.1 502 Bad Gateway
sudo tail -n 3 /var/log/nginx/error.log
sudo systemctl start node-lab
```

Expected error log line:

```
connect() failed (111: Connection refused) while connecting to upstream,
client: 127.0.0.1, server: node-lab.local, upstream: "http://127.0.0.1:3000/"
```

INTERVIEW TRAP

On RHEL with SELinux enforcing, the proxy returns 502 even though Node.js is running. The error log shows `(13: Permission denied) while connecting to upstream`. SELinux blocks Nginx from making network connections by default. Fix it with `sudo setsebool -P httpd_can_network_connect 1`, never by disabling SELinux. Check the mode with `getenforce`.

Task 6: PHP with PHP-FPM

Step	Ubuntu	RHEL-family
Install	`sudo apt install -y php-fpm php-cli`	`sudo dnf install -y php-fpm php-cli` then `sudo systemctl enable --now php-fpm`
Socket	`ls /run/php/` (e.g. `php8.3-fpm.sock`)	`/run/php-fpm/www.sock`
Pool config	`/etc/php/VERSION/fpm/pool.d/www.conf`	`/etc/php-fpm.d/www.conf`

```
sudo mkdir -p /var/www/php-site
echo '<?php echo "PHP is running on Linux\n";' | sudo tee /var/www/php-site/index.php

sudo tee /etc/nginx/sites-available/php-lab >/dev/null <<'EOF'
server {
    listen 80;
    server_name php-lab.local;
    root /var/www/php-site;
    index index.php;

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;   # use YOUR socket from ls /run/php/
    }
}
EOF
sudo ln -s /etc/nginx/sites-available/php-lab /etc/nginx/sites-enabled/php-lab
sudo nginx -t && sudo systemctl reload nginx
curl -H "Host: php-lab.local" http://127.0.0.1/index.php
```

On RHEL, replace the two lines inside the `location` block with: `fastcgi_pass unix:/run/php-fpm/www.sock;`, `include fastcgi_params;` and `fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;`.

Now remove the whole `location ~ \.php$` block, reload, and request the page again. Nginx sends the raw PHP source as a download. On a real site, that leaks code and any credentials inside it. Put the block back afterwards.

> **PRODUCTION WARNING**
>
> Never leave a `phpinfo()` page on a reachable server. It reveals versions, paths, modules and environment variables.

Task 7: Apache next to Nginx

Nginx already owns port 80, so Apache has to use a different one. On Ubuntu, the install may report that Apache failed to start. That is expected.

```
sudo apt install -y apache2
sudo sed -i 's/^Listen 80$/Listen 8081/' /etc/apache2/ports.conf
sudo sed -i 's/<VirtualHost \*:80>/<VirtualHost *:8081>/' /etc/apache2/sites-available/00
sudo apachectl configtest
sudo systemctl restart apache2
sudo dpkg --configure -a          # only if apt reported a half-configured package
curl -I http://localhost:8081
```

On RHEL-family systems: `sudo dnf install -y httpd`, then change `Listen 80` to `Listen 8081` in `/etc/httpd/conf/httpd.conf`. SELinux allows only labelled ports. Check with `sudo semanage port -l | grep '^http_port_t'`, and if 8081 is missing, add it with `sudo semanage port -a -t http_port_t -p tcp 8081` (package `policycoreutils-python-utils`).

Troubleshooting challenges

Symptom	First commands	Typical cause
Nginx returns 502	`tail error.log`, `systemctl status APP`, `ss -tlnp`	Upstream down, wrong port, PHP socket path wrong, SELinux boolean
Node.js service fails to start	`systemctl status`, `journalctl -u node-lab`	`203/EXEC` (wrong path), permissions, port in use
PHP page downloads instead of running	`nginx -T | grep -A5 php`	Missing `location ~ \.php$` block or include
Port 80 already in use	`ss -tlnp 'sport = :80'`	Apache and Nginx both listening on 80
Works on the IP, not the domain	`getent hosts`, `nginx -T | grep server_name`	DNS or `/etc/hosts`, `server_name` typo, default server answering
Deploy user cannot update files	`namei -l`, `id deploy`	Wrong owner or group on the web root (see Lab 12)

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- Nginx vs Apache, and what a reverse proxy adds (TLS termination, buffering, routing).
- How Nginx chooses a server block (`listen`, `server_name`, `default_server`).
- How PHP-FPM works and why Nginx cannot run PHP by itself.
- How to run an application as a hardened systemd service, and how to debug 502 and 504.

12 CI/CD Deployment and Production Troubleshooting

Lab	Level	Revises	Best environment	Time
Lab 12	Advanced	Chapters 10 and 16	VM or EC2 with Nginx (Lab 11 for step 3 of Task 7)	2 hours

LAB OBJECTIVE

Build the Linux side of a CI/CD deployment: a deploy user, verified artifacts, release directories, an atomic symlink switch, a health check and automatic rollback.

Scenario: Your organisation deploys to Linux servers from GitLab CI/CD or Jenkins. The pipeline must fail when the application is broken, not only when a command fails.

Task 1: Prepare the deploy user and directory layout

```
sudo useradd -m -s /bin/bash deploy
sudo mkdir -p /var/www/myapp/releases
sudo chown -R deploy:deploy /var/www/myapp
sudo chmod 755 /var/www/myapp /var/www/myapp/releases
```

```
/var/www/myapp/
├── releases/
│   ├── 20261105-104512/      (v1)
│   └── 20261105-110230/      (v2)
└── current → releases/20261105-110230     (Nginx serves this path)
```

Give the CI system its own SSH key. On the **client or CI runner**:

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy_lab -N "" -C "ci-deploy"   # in real CI, store it
cat ~/.ssh/deploy_lab.pub
```

On the **server**, install the public key:

```
sudo install -d -m 700 -o deploy -g deploy /home/deploy/.ssh
echo "PASTE_PUBLIC_KEY_HERE" | sudo tee /home/deploy/.ssh/authorized_keys
sudo chown deploy:deploy /home/deploy/.ssh/authorized_keys
sudo chmod 600 /home/deploy/.ssh/authorized_keys
sudo -u deploy mkdir -p /home/deploy/incoming
```

With only one VM, use `localhost` as `SERVER_IP`. Optionally, let the deploy user reload Nginx and nothing else: `deploy ALL=(root) NOPASSWD: /usr/bin/systemctl reload nginx` in `/etc/sudoers.d/deploy` (create it with `visudo -f`, as in Lab 2).

Task 2: Point Nginx at the "current" symlink

```
sudo tee /etc/nginx/sites-available/myapp >/dev/null <<'EOF'
server {
    listen 80;
    server_name myapp.local;
    root /var/www/myapp/current;
    index index.html;
}
EOF
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp
# RHEL: /etc/nginx/conf.d/myapp.conf
sudo nginx -t && sudo systemctl reload nginx
```

Task 3: Build versioned artifacts with checksums

```
mkdir -p ~/linux-labs/projects/deployment-demo && cd ~/linux-labs/projects/deployment-dem
for v in v1 v2; do
    mkdir -p "$v"
    echo "<h1>Deployment Successful - $v</h1>" > "$v/index.html"
    echo "$v" > "$v/version.txt"
    tar -czf "myapp-$v.tar.gz" -C "$v" index.html version.txt
    sha256sum "myapp-$v.tar.gz" > "myapp-$v.tar.gz.sha256"
done
ls -l *.tar.gz*
```

Task 4: Install the deploy script on the server

```
sudo -u deploy tee /home/deploy/deploy.sh >/dev/null <<'EOF'
#!/bin/bash
# deploy.sh - release into a new directory, switch atomically, health check, auto-rollbac
set -Eeuo pipefail

ARTIFACT="${1:?Usage: deploy.sh /path/to/artifact.tar.gz}"
APP_DIR="/var/www/myapp"
RELEASES="$APP_DIR/releases"
CURRENT="$APP_DIR/current"
HEALTH_URL="http://127.0.0.1/version.txt"
KEEP=5

# 1. Verify artifact integrity
( cd "$(dirname "$ARTIFACT")" && sha256sum -c "$(basename "$ARTIFACT").sha256" )

# 2. Extract into a new release directory
RELEASE="$RELEASES/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$RELEASE"
tar -xzf "$ARTIFACT" -C "$RELEASE"

# 3. Validate contents BEFORE switching
for f in index.html version.txt; do
    if [ ! -s "$RELEASE/$f" ]; then
        echo "ERROR: $f missing from artifact. Release aborted." >&2
        rm -rf -- "$RELEASE"
        exit 1
    fi
done

# 4. Atomic switch: build the new link, then rename it over the old one
PREVIOUS=$(readlink "$CURRENT" || true)
ln -sfn "$RELEASE" "$CURRENT.new"
mv -Tf "$CURRENT.new" "$CURRENT"
echo "Switched: ${PREVIOUS:-none} → $RELEASE"

# 5. Health check: the live site must serve the version we just deployed
EXPECTED=$(cat "$RELEASE/version.txt")
sleep 1
ACTUAL=$(curl -fsS -H "Host: myapp.local" "$HEALTH_URL" || true)
if [ "$ACTUAL" ≠ "$EXPECTED" ]; then
    echo "Health check FAILED (expected '$EXPECTED', got '$ACTUAL')" >&2
    if [ -n "$PREVIOUS" ]; then
        ln -sfn "$PREVIOUS" "$CURRENT.new"
        mv -Tf "$CURRENT.new" "$CURRENT"
        echo "Rolled back to $PREVIOUS" >&2
    fi
    exit 1
```

```
fi
echo "Health check passed: version $ACTUAL"

# 6. Keep only the newest $KEEP releases
ls -1dt "$RELEASES"/*/ | tail -n +$((KEEP + 1)) | xargs -r rm -rf --
EOF
sudo chmod 750 /home/deploy/deploy.sh
```

Task 5: Transfer and deploy (what the pipeline does)

```
cd ~/linux-labs/projects/deployment-demo
scp -i ~/.ssh/deploy_lab myapp-v1.tar.gz myapp-v1.tar.gz.sha256 deploy@SERVER_IP:incoming
ssh -i ~/.ssh/deploy_lab deploy@SERVER_IP './deploy.sh incoming/myapp-v1.tar.gz'
echo "pipeline exit code: $?"
```

Expected:

```
myapp-v1.tar.gz: OK
Switched: none → /var/www/myapp/releases/20261105-104512
Health check passed: version v1
pipeline exit code: 0
```

Deploy `v2` the same way, then verify:

```
ls -l /var/www/myapp/
curl -s -H "Host: myapp.local" http://127.0.0.1/
```

DEVOPS EXAMPLE

The same two commands form a GitLab CI deploy job. `DEPLOY_KEY` is a file-type CI variable. Pin the server's host key in `known_hosts` instead of disabling host key checking.

```
deploy:
  stage: deploy
  script:
    - scp -i "$DEPLOY_KEY" myapp.tar.gz myapp.tar.gz.sha256 deploy@$SERVER:incoming/
    - ssh -i "$DEPLOY_KEY" deploy@$SERVER "./deploy.sh incoming/myapp.tar.gz"
```

Because `deploy.sh` exits 1 when the health check fails, the pipeline turns red when the application is broken, not just when a command fails.

Task 6: Why a symlink switch?

Approach	During the deploy	Rollback
Copy over the live directory	Users can see a mix of old and new files	Restore from backup (slow)
Release directories + symlink	The `mv -T` rename is atomic: old or new, never half	Point the link back (seconds)

`ln -sfn` on its own removes and recreates the link, which leaves a tiny window with no link. Creating `current.new` and renaming it with `mv -T` avoids that window. PHP-FPM caches resolved paths, so PHP sites behind a symlink should use `$realpath_root` in `SCRIPT_FILENAME`.

Task 7: Simulate failed deployments

```
cd ~/linux-labs/projects/deployment-demo

# 1. Incomplete artifact: caught BEFORE the switch
tar -czf myapp-bad1.tar.gz -C v2 index.html
sha256sum myapp-bad1.tar.gz > myapp-bad1.tar.gz.sha256

# 2. Permission bug: passes validation, fails the health check → automatic rollback
mkdir -p v3 && echo "<h1>v3</h1>" > v3/index.html && echo v3 > v3/version.txt
chmod 600 v3/version.txt          # the Nginx worker cannot read it: 403
tar -czf myapp-v3.tar.gz -C v3 index.html version.txt
sha256sum myapp-v3.tar.gz > myapp-v3.tar.gz.sha256
```

Deploy both artifacts with the same `scp` and `ssh` commands as in Task 5. Confirm that the site still serves `v2` each time.

Then practise two server-side failures, and undo each one afterwards:

3. Change `proxy_pass` in the `node-lab` site (Lab 11) to port `3001` and reload. Result: 502.
4. Add an invalid directive to a test file in `conf.d`. Result: `nginx -t` fails, and the reload is refused.

```
sudo nginx -t
sudo systemctl status nginx --no-pager
sudo journalctl -u nginx -n 100 --no-pager
sudo ss -tlnp
curl -v -H "Host: node-lab.local" http://127.0.0.1/
sudo tail -n 20 /var/log/nginx/error.log
```

Troubleshooting challenge

Challenge 1 **"The pipeline is green, but the site returns 502"**

Reproduce it: deploy successfully, then stop the backend. Write an investigation report that covers every row:

Area	Command	What you record
Pipeline	CI job log	Which commands ran, and which exit codes
Artifact	`sha256sum -c`, `tar -tzf`	Integrity and contents
Release	`ls -l /var/www/myapp/`	Where `current` points
Permissions	`namei -l`	Can the web user read every path component?
Application service	`systemctl status`, `journalctl -u`	State, crashes, restarts
Listening port	`ss -tlnp`	Port and bind address
Nginx upstream	`nginx -T`, `error.log`	Upstream address and the exact error
Decision	–	Fix forward or roll back, and why

INTERVIEW TRAP QUESTION

The deployment pipeline succeeded. Does that mean the application is working?

No. A pipeline only knows the exit codes of the commands it ran. Copying files can succeed while the service is down, the port is wrong, permissions block the web user, or the application crashes on start. That is why the deploy step must end with a real health check that fails the pipeline and triggers a rollback.

INTERVIEW QUESTION

After this lab, can you explain these without notes?

- How a pipeline deploys to Linux over SSH, and how the deploy key is protected.
- Why you use a dedicated deploy user with minimal sudo.
- Artifact integrity, release directories, atomic symlink switching and rollback.
- How rolling and blue/green deployments extend this idea across many servers.

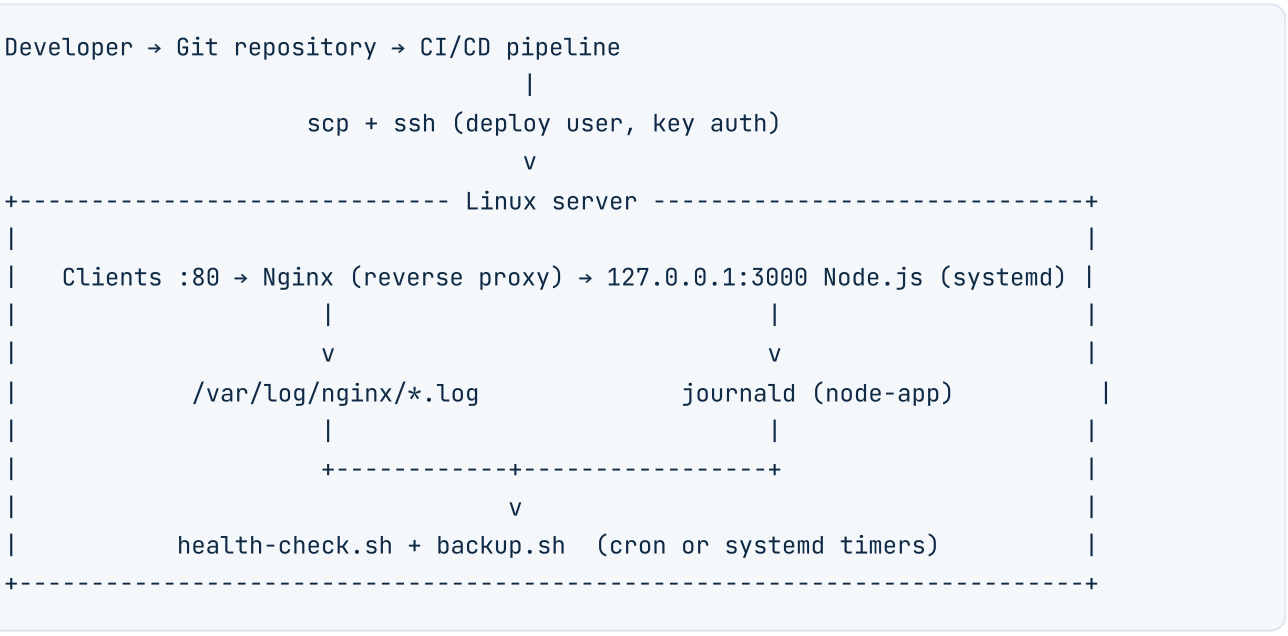
13 Capstone Project: Build a Production-Style Linux Server

Project	Level	Revises	Best environment	Time
Capstone	Advanced	Chapters 1–18	Fresh VM or EC2 instance	1–2 days

LAB OBJECTIVE

Combine everything in one project. Start from a fresh server and finish with a secured, monitored Node.js application behind Nginx, with automated backups and a deployment process that can roll back. Use the labs only as a reference. The aim is to work from memory.

Project architecture



Build phases

Phase 1 Server preparation

- Create the VM or EC2 instance, apply updates and set the hostname.
- Configure key-only SSH and disable root login (Lab 5).
- Create the `deploy` user with limited sudo (Labs 2 and 12).
- Enable the firewall for 22 and 80 only; on EC2, configure the security group as well.

Done when: you can SSH in with a key, password login is refused, and only the expected ports are open (`ss -tlnp`).

Phase 2 Application deployment

- Install Node.js LTS and run the app as a systemd service under a dedicated user (Lab 11).
- Configure Nginx as the reverse proxy and verify with `curl` and a browser.

Done when: the app answers through Nginx and survives a reboot.

Phase 3 Security

- Disable services you do not need (`systemctl list-unit-files --state=enabled`).
- Set least-privilege permissions on the application and web directories.
- Keep secrets out of the code: use an `EnvironmentFile=` with mode 600, or AWS SSM Parameter Store or Secrets Manager.
- Enable automatic security updates, and Fail2ban if SSH is exposed (Chapter 9).

Done when: you can justify the owner and mode of every application path.

Phase 4 Automation

- Schedule the health-check script (disk, memory, load, services) and the backup script (Lab 6).
- Prove that a restore works, and add backup retention.

Done when: scheduled runs appear in the logs, and a failing check shows up as a failed unit or a non-zero exit code.

Phase 5 Monitoring and logs

- Know where every log lives: system, Nginx and application.
- Configure log rotation, and on EC2 optionally send logs and metrics to CloudWatch.
- Set up at least one alert: an email from a script, or a CloudWatch alarm on disk or CPU.

Done when: you can answer "how many 5xx errors in the last hour?" in one command.

Phase 6 Deployment and rollback

- Package the app as a versioned artifact with a checksum.
- Deploy with release directories and a symlink switch, run a health check, and roll back automatically (Lab 12).

Done when: a deliberately broken release is rolled back with no manual steps.

Phase 7: Troubleshooting assessment

Ask a partner to break the server, or break it yourself in a separate session and wait an hour. Then investigate without notes. For every scenario, record the commands you ran, the evidence you found, the root cause and the fix.

Scenario 1 Nginx is stopped

Break: `sudo systemctl stop nginx`. **Prove:** connection refused, `systemctl status` inactive, nothing on :80.

Scenario 2 The Node.js application is stopped

Break: stop the app service. **Prove:** 502 from Nginx, `111: Connection refused` in `error.log`.

Scenario 3 Port 3000 is not listening

Break: a drop-in setting `Environment=PORT=3005`. **Prove:** `ss -tlnp` shows 3005, and the Nginx upstream still points at 3000.

Scenario 4 Nginx returns 502 with the app running

Break: a typo in the `proxy_pass` port (or, on RHEL, `setsebool httpd_can_network_connect 0`). **Prove:** the exact error in `error.log` and the upstream shown by `nginx -T`.

Scenario 5 Deployment fails with permission denied

Break: `sudo chown root:root /var/www/myapp/releases`. **Prove:** the deploy error, then `namei -l` and `id deploy`.

Scenario 6 Disk usage reaches 95%

Break: mount a 512 MB loop filesystem as the app's upload directory and fill it with `fallocate`. Never fill the real root disk. **Prove:** `df -h`, `du`, and a check for deleted-but-open files.

Scenario 7 Application logs show repeated errors

Break: a loop of requests to a failing path. **Prove:** an `awk` / `sort` / `uniq` summary showing count, path, source IP and time window.

Scenario 8 The service fails after reboot

Break: `sudo systemctl disable APP`, then reboot. **Prove:** `systemctl is-enabled`, and `journalctl -b` with no start attempt.

Scenario 9 DNS resolves to the wrong IP

Break: an `/etc/hosts` entry pointing the site name at a wrong IP. **Prove:** `getent hosts` and `dig` disagree; `nsswitch.conf` order.

Scenario 10 A new deployment breaks the application

Break: deploy the `v3` permission-bug artifact. **Prove:** the deploy script output, the rollback, and that the old version is serving again.

Capstone deliverables

- A `README.md` runbook: architecture, how to deploy, how to roll back, where the logs are.
- All scripts and unit files in a Git repository, passing `shellcheck`.
- Ten short incident reports, one per scenario.

14 Final Assessment

Tick a skill only after you have done it yourself, without step-by-step help. Recognising a command does not count.

Skill	Practical proof	Lab	Done
Linux basics	Navigate, manage files, explain links and inodes	1	☐
Users and permissions	Shared team directory with setgid and a default ACL; restricted sudo	2	☐
Services	Install, enable and debug a systemd service; safe reload	3	☐
Storage	Create, mount and persist a filesystem; fix disk-full and inode issues	4	☐
Networking	Debug DNS, ports and SSH; tell refused from timed out	5	☐
Bash	Health-check and backup scripts with correct exit codes, scheduled	6	☐
Logs	Summarise access logs and the journal in one command each	7	☐
Performance	Identify CPU vs memory vs I/O bottlenecks with evidence	8	☐
Docker	Show namespaces and cgroups; debug exits, mounts and OOM	9	☐
Kubernetes	Deploy a workload; debug CrashLoopBackOff and ImagePullBackOff	9	☐
AWS	EC2 with IMDSv2, SSM, an EBS volume grown online, S3 via a role	10	☐
Deployment	Node.js and PHP behind Nginx; explain and fix a 502	11	☐
CI/CD	Release with health check and automatic rollback	12	☐
Production troubleshooting	All ten capstone scenarios solved, with written reports	13	☐

Suggested completion levels

Level	Completed	What it shows
Beginner	Labs 1–3	You can operate a Linux server day to day
Intermediate	Labs 1–7	You can administer and automate a server
Advanced	Labs 1–12	You can run applications in production, on-premises and on AWS
Job-ready foundation	Labs 1–12 + capstone, done independently	You can build, deploy, monitor and troubleshoot without a tutorial

These are personal learning milestones, not industry certifications. In interviews, the evidence that counts is your ability to explain what you did and why.

15 Workbook Takeaways and Next Steps

- ✓ Read the error first. Most incidents are solved by the exact message in `journalctl`, `error.log` or `nginx -t`.
- ✓ Prove before you change: back up the config, validate it (`nginx -t`, `sshd -t`, `findmnt --verify`), then reload.
- ✓ "Refused" and "timed out" point to different layers. Know which commands test each one.
- ✓ Disk problems come in three kinds: space, inodes, and deleted-but-open files.
- ✓ `available` memory matters, not `free`. High load can be I/O, not CPU.
- ✓ Containers are Linux processes with namespaces and cgroups. Debug them with Linux tools.
- ✓ On AWS, check the cloud layer (security groups, NACLs, roles, AZs) and the Linux layer.
- ✓ Scripts and pipelines must fail loudly. A green pipeline with a broken site is worse than a red one.
- ✓ Every change needs a rollback plan you have actually tested.

PRACTICAL EXERCISES

Repeat this loop for every lab until you no longer need this workbook:

1. Understand what each command does before you run it.
2. Execute it yourself, typing rather than pasting.
3. Observe and read the output.
4. Introduce a controlled error.
5. Troubleshoot it using evidence, not guesses.
6. Document the cause and the fix.
7. Explain the task out loud in your own words, as in an interview.

Your final goal: log in to a Linux server, configure it, deploy an application, automate administration, monitor its health and troubleshoot common production issues, all without following a tutorial step by step.

WHAT'S NEXT

Linux Cheatsheets: Quick-Reference Series

Separate one-page references for commands, permissions, networking, storage, scripting, monitoring, troubleshooting and interviews, built for last-minute revision before an interview.

END OF LAB WORKBOOK